

MÁSTER UNIVERSITARIO EN TECNOLOGÍAS WEB,
COMPUTACIÓN EN LA NUBE Y
APLICACIONES MÓVILES



VNIVERSITAT
DE VALÈNCIA

TRABAJO FIN DE MÁSTER

MEDICIÓN DE CONSUMO ENERGÉTICO DE
CARGAS TRABAJO DE CODIFICACIÓN DE VIDEO
SOBRE CONTENEDORES

AUTOR: GUILLERMO BLÁZQUEZ ORTEGA

TUTORA: JUAN GUTIERREZ AGUADO

JULIO 2026

Declaración de autoría:

Yo, Guillermo Blázquez Ortega, declaro la autoría del Trabajo Fin de Máster titulado “Medición de Consumo Energético de cargas trabajo de codificación de video sobre contenedores” y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores.

Firmado:

Resumen:

Conocer el consumo energético que una carga de trabajo necesita es un aspecto esencial en el contexto actual, en el cual la mayoría de la carga de computación se lleva a cabo en grandes CPDs empleando arquitectura de contenedores. Una toma de métricas constante permite realizar estimaciones precisas, aportando mucha información relevante a la hora de definir diferentes aspectos técnicos.

El siguiente Trabajo Fin de Máster gira en torno a esta premisa y a las problemáticas que ello implica, estando enfocado en la codificación de video de alta resolución sobre contenedores que emplean CPU y GPU física para acelerar el proceso. El objetivo es lograr la mejor configuración de codificación de video para FFmpeg que consuma la menor cantidad de energía sin empobrecer la calidad. Para poder lograrlo, se ha requerido una herramienta que permita medir el consumo de procesos individuales, tanto sobre la CPU como en la GPU, y para ello se ha realizado una comparativa entre herramientas, las cuales están pensadas para medir el consumo de trabajo de tareas desplegadas sobre contenedores.

Una vez se ha seleccionado la herramienta indicada, se han realizado un conjunto de baterías de pruebas que me han permitido llegar a las CONCLUSIONES.

Abstract:

This is the abstract of the TFM. It must be short and cover the main aspects of the TFM. esto es una prueba de escritura usando el

Resum:

Aquest és el resum del TFM. Ha de ser curt (màxim mitja pàgina) i cobrir els aspectes principals del TFM.

Agradecimientos:

Me gustaria agrader el apollo de mi tutor Juan Gutierrez Aguado, quien me ha acompañado en todo el proceso de elaboracion y redaccion del siguiente trabajo fin de master. Gracias a Juan, he podido emplear referencias de valor y definir pruebas que demuestren la relevancia de mi trabajo.

Índice general

1. Introducción	9
1.1. Contexto del Trabajo	9
1.2. Motivaciones y Objetivos	11
2. Estado del Arte	13
2.1. Contexto Global Consumo Energetico	13
2.1.1. Generacion Energetica	14
2.1.2. Eficiencia Energetica	14
2.2. Especificaciones Tecnicas del Entorno de Pruebas	14
2.3. Selecccion de Herramientas de Consumo Energetico	15
3. Desarrollo de Proyecto	19
3.1. Configuración de Entorno & Herramientas	19
3.1.1. Selecccion e Intalacion del Sistema Operativo	19
3.1.2. Configuración e Instalación de Herramientas	21
3.1.3. Estado Final del Sistema	25
3.2. Selección de Herramienta para la toma de Métricas de Consumo	26
3.2.1. Desarrollo de los experimentos de la primera fase	26
3.2.2. Post-procesamiendo de las Métricas de Consumo Energético	27
3.2.3. Analisis de los resultados obtenido en cada uno de los escenarios	27
3.2.4. Conclusiones fase 1 de la experimentacion	33
3.3. Consumo energetico en relacion al SITI y el Numero de Threas	34
3.3.1. Clasificacion y Selecccion de Chunks en base al SITI	36
3.3.2. Consumo energetico en relacion al SITI	37
3.3.3. Relación entre numero de cores, tiempo de codificación y consumo energetico	37
4. Pruebas y Resultados	39
5. Conclusiones y Trabajo Futuro	41

A. Apéndice	43
A.1. Subapendice: Experimentacion	43
A.1.1. Experimentación Fase 1	44
A.1.2. Experimetacion Fase 2	59
Bibliografia	59

Capítulo 1

Introducción

1.1. Contexto del Trabajo

En la actualidad, la carga computacional ha migrado del dispositivo local hacia grandes Centros de Procesamiento de Datos (CPD), los cuales emplean una arquitectura de microservicios basada en contenedores y gestionada por orquestadores como Kubernetes. Este cambio de paradigma ha permitido aumentar la capacidad de los servicios y aplicaciones para dar servicios a múltiples nuevos clientes, así como la ejecución simultánea de tareas de alta computación en paralelo, las cuales antes estaban limitadas por la capacidad hardware del proveedor. Como consecuencia, el consumo de recursos se ha visto altamente afectado, siendo uno de los más afectados el consumo energético, el cual, según se indica en [1], supone entre el 15 y el 20 % del consumo global energético.

Siendo más precisos, basándome en el artículo [2], sabemos que la navegación por internet supone un 3.7 % de los gases de efecto invernadero, del cual el 65 % corresponde al streaming de video. Esto supone que el streaming de video es responsable de las emisiones de efecto invernadero equivalentes a las liberadas por la industria de transporte aéreo, como se cita en el paper [2].

Es por este motivo que la obtención de métricas precisas, segmentadas por proceso y por área de hardware utilizada, se ha convertido en una necesidad vital. Tanto en el ámbito de la estimación de usos y costes como en la gestión de análisis de sostenibilidad, siendo estos últimos esenciales para la toma de decisiones a futuro, tanto en el contexto industrial como en el contexto socialpolítico.

Bajo este contexto, se desarrolla el siguiente Trabajo de Fin de Máster (TFM), siguiendo una de las líneas de investigación del proyecto Energy-Conscious Optimization for Cloud-Edge Video Ecosystems (ECO-STREAM). Este proyecto tiene como objetivo la optimización de cargas de trabajo de codificación y streaming de vídeo de alta calidad, en arquitectura de contenedores, mediante el uso de CPU y GPU físicas.

El objetivo principal de este trabajo de fin de máster es definir la configuración de codificación de alta resolución que minimice el consumo energético sin comprometer la calidad del resultado final. Para ello, es indispensable contar con herramientas capaces de medir el consumo energético a nivel de contenedor de forma precisa, segmentando el gasto entre CPU y GPU.

Para la adquisición de métricas, se propone el uso de herramientas software adaptadas para el contexto de las cargas de trabajo, utilizando la medición de consumo energético

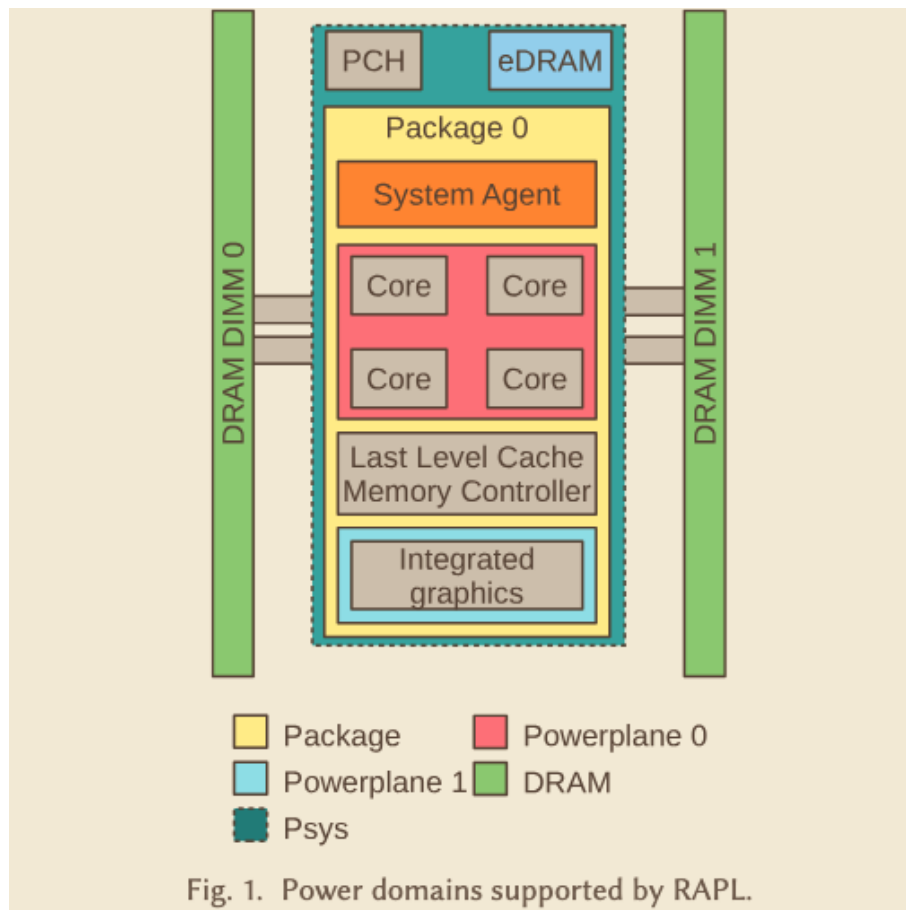


Figura 1.1: Agrupación de componentes en metrica de RAPL

total del sistema como punto de comparación y verificación. El uso de herramientas hardware se ha descartado debido a su complejidad de implementación, así como a los desafíos de mantenimiento y escalabilidad que presentan en entornos de producción.

Las herramientas seleccionadas utilizarán Running Average Power Limit Energy Reporting (RAPL) [3], una aplicación de Intel que permite extraer métricas precisas del consumo energético de varios componentes del sistema, junto con la API interna de NVIDIA y sus librerías asociadas. Cabe destacar que RAPL agrupa en grupos y subgrupos los componentes del sistema, como se observa en la imagen 1.1, permitiendo obtener métricas precisas.

Una vez definido el contexto y la problemática que se desea solventar en la introducción 1, así como los objetivos y motivaciones del proyecto. Posteriormente, en el capítulo 2 (Estado del Arte) se definirá el marco teórico y contexto tecnológico, el cual será la base con la que justificaremos las decisiones tomadas a lo largo de este trabajo. Seguidamente, en el capítulo 3 (Desarrollo del Proyecto) se describirán los diferentes pasos, configuraciones y problemáticas encontradas durante el desarrollo del trabajo. En el capítulo 4 (Resultados y Pruebas), se hará uso de los datos y resultados obtenidos del apartado anterior y se agregarán diferentes tablas y ayudas gráficas, las cuales facilitarán la comprensión y el desarrollo de las conclusiones. Para finalizar, en el capítulo 5 (Conclusiones y Trabajo Futuro), se expondrán las conclusiones y reflexiones acerca del trabajo realizado. Destacar que al final de este trabajo también se podrá encontrar la bibliografía A.1.2 y un anexo A, el cual contendrá información adicional.

1.2. Motivaciones y Objetivos

El presente trabajo se centra en la identificación de configuraciones de codificación de vídeo energéticamente eficientes mediante el uso de FFmpeg sobre entornos basados en contenedores. La gran cantidad de parámetros disponibles durante el proceso de codificación dificulta la selección de aquellas configuraciones capaces de proporcionar una elevada calidad visual manteniendo un consumo energético reducido. Esta problemática adquiere una especial relevancia en entornos cloud y de computación distribuida, donde pequeñas mejoras en la eficiencia pueden traducirse en reducciones significativas del consumo energético y de los costes operativos.

Para abordar este problema, resulta necesario disponer de mecanismos que permitan medir de forma precisa el consumo energético asociado a cada carga de trabajo. Sin embargo, las herramientas existentes presentan limitaciones en términos de precisión, granularidad o capacidad de integración con entornos basados en contenedores. Por este motivo, una parte fundamental del trabajo consiste en el análisis de las soluciones disponibles y en el desarrollo de una herramienta propia capaz de monitorizar con precisión el consumo energético de los recursos hardware involucrados, especialmente la CPU y la GPU.

La motivación principal de este trabajo se encuentra alineada con los objetivos del proyecto ECO-STREAM, cuyo propósito es mejorar la eficiencia energética de los procesos de codificación y transmisión de vídeo. La optimización de estos procesos permite un mejor aprovechamiento de los recursos computacionales, una reducción del consumo energético y, en consecuencia, una disminución de la huella de carbono asociada a las infraestructuras de procesamiento multimedia.

Asimismo, este trabajo pretende aportar una metodología reproducible para la evaluación energética de cargas de trabajo de codificación de vídeo, así como información de utilidad para futuras investigaciones relacionadas con la monitorización energética, la optimización de aplicaciones multimedia y el desarrollo de infraestructuras cloud más sostenibles.

Capítulo 2

Estado del Arte

2.1. Contexto Global Consumo Energetico

Bajo el paradigma actual de la computación distribuida pueden distinguirse dos modelos principales de despliegue en la nube [4]: las nubes públicas y las nubes privadas. Las primeras ofrecen recursos de computación como un servicio accesible para terceros, permitiendo a los clientes utilizar la capacidad de procesamiento de grandes Centros de Procesamiento de Datos (CPDs) sin necesidad de gestionar ni mantener la infraestructura subyacente. Por el contrario, las nubes privadas son administradas por la propia organización sobre una infraestructura dedicada, lo que implica asumir costes adicionales asociados al mantenimiento del hardware, la administración de los sistemas y el consumo energético necesario para garantizar su funcionamiento.

La creciente adopción de estos modelos de computación ha impulsado la construcción de grandes infraestructuras capaces de atender millones de peticiones de forma simultánea. Entre ellas destacan las Redes de Distribución de Contenido (CDN, Content Delivery Networks), cuya finalidad es distribuir contenido multimedia a través de redes de alta velocidad para reducir la latencia y mejorar la experiencia de los usuarios. Tanto los CPDs como las CDN requieren una gran cantidad de recursos energéticos para mantener un funcionamiento continuo las 24 horas del día, los 365 días del año [5]. Este consumo no se limita únicamente a los equipos de procesamiento, sino que también incluye los sistemas de refrigeración necesarios para mantener unas condiciones de operación adecuadas, siendo el agua uno de los recursos más utilizados en la industria para este fin

El incremento del número de CPDs y de la capacidad de procesamiento requerida ha supuesto un importante impacto desde el punto de vista medioambiental. Según el informe publicado por la International Energy Agency (IEA) [6], la demanda energética asociada a los centros de datos continúa aumentando debido al crecimiento de las cargas de trabajo digitales, lo que ha impulsado la necesidad de ampliar la capacidad de generación eléctrica [7]. Entre los principales factores responsables de este incremento destacan el auge de la inteligencia artificial, el procesamiento masivo de datos y la construcción de nuevas infraestructuras tecnológicas, como centros de datos y redes de distribución de contenido, que requieren un suministro energético continuo y sistemas de refrigeración de gran capacidad.

Como consecuencia de esta elevada demanda, numerosos proyectos tecnológicos se enfrentan actualmente a limitaciones en el acceso a la red eléctrica. El informe de la IEA indica que existen solicitudes de conexión que representan aproximadamente 2.500 GW

de nueva demanda eléctrica, muchas de las cuales permanecen en espera debido a la insuficiente capacidad de generación y distribución energética. Aunque el propio informe señala que parte de estas solicitudes corresponden a proyectos que finalmente podrían no materializarse, pone de manifiesto que el crecimiento de la demanda energética constituye un desafío real y de alcance global.

2.1.1. Generacion Energetica

Como respuesta al incremento de la demanda energética y a la necesidad de reducir el impacto medioambiental, durante los últimos años se han producido cambios significativos tanto en las fuentes de generación eléctrica como en su distribución. Según el informe publicado por la International Energy Agency (IEA) [7], la tendencia a medio y largo plazo apunta hacia un aumento progresivo de la participación de fuentes de energía renovables y de tecnologías de bajas emisiones. Entre ellas, la energía solar fotovoltaica destaca como la tecnología con mayor crecimiento a nivel mundial, impulsada principalmente por las inversiones realizadas por China y Estados Unidos durante los últimos años. Este crecimiento ha permitido reducir la dependencia del carbón en numerosos sistemas eléctricos, incluso en un contexto de aumento de la demanda energética. Asimismo, la biomasa continúa consolidándose como una alternativa renovable relevante debido al aprovechamiento energético de residuos, mientras que la energía nuclear ha alcanzado niveles históricos de generación eléctrica, contribuyendo a satisfacer el incremento de la demanda mundial.

No obstante, los combustibles fósiles continúan desempeñando un papel predominante en el sistema energético global. Aproximadamente el 35 % de la energía consumida procede del petróleo y alrededor del 20 % del gas natural, que continúa siendo una de las principales fuentes de generación eléctrica a nivel mundial. Aunque el incremento de las energías renovables y las mejoras en eficiencia energética han permitido limitar el crecimiento de las emisiones de dióxido de carbono, las emisiones globales alcanzaron aproximadamente 38,400 millones de toneladas de CO₂, impulsadas principalmente por el uso de carbón, gas natural y diversos procesos industriales [7].

Ante este escenario, la expansión de las fuentes de generación con bajas emisiones constituye uno de los principales objetivos de las políticas energéticas internacionales. En este sentido, la COP [8] establece como uno de sus objetivos incrementar de forma significativa la participación de las energías renovables y de la energía nuclear en el mix energético antes de 2030, al mismo tiempo que se continúa reduciendo la dependencia del carbón. Para alcanzar estas metas también será necesario modernizar las infraestructuras eléctricas mediante el aumento de la capacidad de transporte, almacenamiento y distribución de energía. Estas medidas permitirán integrar una mayor proporción de energías renovables, satisfacer el crecimiento de la demanda eléctrica y contribuir a una reducción progresiva de las emisiones globales de gases de efecto invernadero.

2.1.2. Eficiencia Energetica

2.2. Especificaciones Tecnicas del Entorno de Pruebas

En cuanto al entorno en el que se ha llevado a cabo el experimento, se ha utilizado un ordenador con una CPU Intel i7-4790K y 8 cores. Esta CPU es relativamente actual y nos

permite tomar medidas de casi cualquier parte del sistema utilizando RAPL [9]. También cuenta con una memoria RAM de 32 GB y 2 TB de memoria física HDD.

La tarjeta gráfica seleccionada es una NVIDIA Quadro M4000 [10], elegida entre el conjunto de GPUs disponibles en el laboratorio. En primer lugar, la tarjeta es compatible con la interfaz NVIDIA System Management (NVIDIA-SMI) [11], herramienta empleada para la monitorización y extracción de métricas relacionadas con el estado y el consumo energético de la tarjeta gráfica. Además, también ofrece soporte para las versiones de CUDA y NVIDIA Container Toolkit [12], lo cual permite el acceso a la GPU desde contenedores y la ejecución de cargas de trabajo con aceleración de hardware. Esta tarjeta gráfica incluye soporte para la tecnología NVENC, la cual permite usar los codificadores de hardware H.264 (AVC) y H.265 (HEVC) en diferentes formatos de codificación. En la Tabla 2.2 se muestran los formatos compatibles:

Codificador	Formato	CPU	GPU
H.264	YUV 4:2:0	x	X
H.264	YUV 4:4:4	X	X
H.264	Lossless	X	X*
H.265	4K YUV 4:2:0	X	X

Cuadro 2.1: Matriz de Capacidad de Codificación de Video de NVIDIA Quadro M4000

El SO sobre el que se han llevado a cabo las pruebas es un Debian 13.1-amd64 de 64 bits (x86_64) con la versión de kernel 6.12.90-2. Sobre este sistema se han instalado los drivers de NVIDIA con la versión 580.159.04 y la versión de CUDA 13.0. Se ha decidido instalar este sistema operativo debido principalmente a su estabilidad y su flexibilidad, habiendo redactado todo el proceso de instalación y preparación del entorno en el subapartado 3.1.3.

2.3. Selección de Herramientas de Consumo Energetico

Las herramientas encargadas de llevar a cabo la toma de métricas de consumo de las cargas de codificación desplegadas sobre contenedores de Docker tendrían que estar enfocadas a entornos de altas prestaciones compuestos por múltiples nodos. Además, debería ser capaz de tomar métricas de consumo de recursos hardware implicados, como la GPU, y poder diferenciar qué porcentaje de recursos ha sido utilizado por el contenedor.

Teniendo en cuenta estas necesidades, se ha realizado una búsqueda entre diversos repositorios, fuentes académicas y papers, como PAPER-KEPLER, y se ha llevado a cabo una selección de las herramientas que más se mencionan, tanto de forma directa como en la bibliografía. Estas dos herramientas han sido: Kepler [13] y Scaphandre [14].

Además, también se han proporcionado dos herramientas por parte de Juan Gutiérrez Aguado, uno de los directores del proyecto Eco-Stream: EnergyMeter, empleada como referencia de consumo energetico del sistema basado en las metricas del sistema obtenidas por RAPL; Y Monitor, una herramienta desarrollada por Juan Gutiérrez Aguado empleada como alternativa a las herramientas anteriores, la cual permite medir el consumo de una carga de trabajo desplegada sobre uno o varios contenedores de forma simultanea y muy precisa.

<i>HERRAMIENTA</i>	<i>NODO</i>	<i>PROCESO</i>	<i>CPU</i>	<i>GPU</i>
<i>Energy Meter</i>	X	-	X	-
<i>Kepler</i>	X	X	X	X
<i>Scaphandre</i>	X	X	X	-
<i>Monitor</i>	X	X	X	-

Cuadro 2.2: Capacidad de toma de metricas de las Herramientas

Kepler

Kubernetes Efficient Power Level Exporter (Kepler) es una herramienta de monitorización de consumo de energía pensada para ser utilizada en entornos con multiples nodos gestionados por Kubernetes sobre los cuales se pretende medir el consumo energetico de una carga de trabajo desplegada sobre un contenedor, y saber cual es el consumo energetico que este presenta.

Se basa en Intel RAPL para la obtencio de metricas de consumo del sistema y se apoya en nvidia-smi y sus librerias internar para obtener metricas de consumo de la GPU. Toma metricas cada 1 segundo, por defecto, y posteriormente aplica un post-procesamiento que resulta en un archivo compatible con Prometheus. Este es un software de monitorizacion de alertas compatible con Grafana, el cual no es necesario utilizar para tomar metricas con Kepler.

El conjunto de datos que proporciona Kepler en la salida se divide en tres bloques: El primero hace relacion al consumo energetico y metricas del programa, el segundo al consumo energetico a nivel de nodo y el tercero al consumo del proceso individual. Tanto en el segundo como el tercer bloque, distinguen metricas por zona de metricas(equivalentes a las que establece RAPL) y por core de la CPU.

Cabe destacar que pese a que Kepler es capaz de identificar que nodos han estado implicados en el proceso, no es capaz de diferenciar acerca del porcentaje de implicacion que cada nodo a tenido en el proceso, reparte el consumo energetico de forma equitativa.

Los resultado que se obtiene del posprocesamiento con kepler, se dividen en dos tipos: Enrgia (Energy), representada en microjulios de forma aculativa el consumo energetico del hardward; Y potencia (Power), representada en microwatios el consumo del hardware a lo largo del tiempo de medicion.

Segun se advierte en la documentacion oficial, Kepler presenta dificultades para medir de forma precisa el consumo energetico en entornos de carga mixta con patrones variables de cómputo y E/S, el modelo de atribución proporcional al tiempo de CPU puede ser inexacto, ya que no distingue entre el alto consumo de un proceso de cómputo intensivo y el menor gasto de uno que pasa gran parte del tiempo esperando operaciones de E/S, a pesar de que ambos puedan tener el mismo uso de CPU. Tambien presenta dificultades para realizar estimaciones sobre cargas de trabajo de alto rendimiento que usan instrucciones especializadas, como las vectoriales, la atribución basada únicamente en el tiempo de CPU no refleja el consumo energético real, que puede ser significativamente mayor. Por lo tanto, Kepler no es adecuado para decisiones de presupuesto energético absoluto ni para una optimización de grano fino que requiera mediciones precisas por proceso, ya que está diseñado para comparaciones relativas y análisis de tendencias, no para métricas exactas

a nivel de proceso individual.

Scaphandre

Scaphandre es un agente de monitorización especializado en la toma de métricas de consumo energético, diseñado por la empresa Hubblo para ayudar a organizaciones e individuos a cuantificar el consumo eléctrico de sus servicios tecnológicos, con un enfoque en la sostenibilidad. Su despliegue es versátil, pudiendo ejecutarse en múltiples entornos de múltiples formas. En sistemas GNU/Linux, se puede instalar desde los repositorios oficiales o mediante paquetes `*.deb*`, así como la posibilidad de desplegar un contenedor Docker que monitorice el consumo energético de un sistema y sus procesos. Además, Scaphandre puede compilarse con un conjunto limitado de características, lo que permite, por ejemplo, instalar únicamente el exportador para Prometheus Push Gateway o instalarse empleado Helm en entornos gestionada por Kubernetes.

La salida de la herramienta Scaphandre puede presentarse en formato `stdout` del sistema, JSON o según el esquema compatible con Prometheus. Dicho archivo contiene las métricas ofrecidas por RAPL en cada instante al iniciar la recolección, en base al valor de monitorización y la duración total de la métrica, los cuales son definidos al lanzar la toma de métricas con la herramienta. Este contiene las métricas del sistema en cada instante, pudiendo diferenciar entre consumo del sistema y consumo de un proceso. Cabe destacar que el conjunto de datos que se muestra representa el consumo energético en el instante de la toma de la métrica y que, por tanto, se repite n veces, dependiendo de cada cuánto se haya definido la toma de estos valores y la duración total de la monitorización, lo cual implica que se han de realizar las operaciones pertinentes a cada métrica en el postprocesado de los datos.

En cuanto a la toma de métricas, nos encontramos con algunas limitaciones; la herramienta emplea RAPL (Running Average Power Limit) para obtener datos de consumo energético, pero no puede acceder a `Nvidia_SMI` (System Management Interface) para medir el consumo de GPU. También presenta fuertes dependencias sobre el sistema operativo: requiere un entorno Linux y, en caso de utilizar procesadores AMD, una versión del kernel superior al 5.11. El uso de esta herramienta en nubes públicas se ve restringido porque obliga a modificar el hipervisor bajo el cual se lanzan las cargas de trabajo, las aplicaciones y las máquinas virtuales —una opción que la mayoría de los proveedores evita para evitar agotar su infraestructura al completo.

Energy Meter

Energy Meter es una de las herramientas desarrollada por el tutor de la Tesis Final de Master Juan Gutierrez Aguado, la cual tiene la capacidad de tomar métricas del consumo de todo el sistema empleando la librería PyRAPL. El formato de salida consiste en un registro JSON que contiene; el consumo energético del paquete (`pkg`), del apartado `*data*` y la suma total de los dos apartados anteriores.

Su principal valor radica en su bajo consumo, ya que opera únicamente consultando a RAPL al iniciarse la toma de métricas y al indicar su finalización. Además, se despliega fácilmente dentro de un contenedor y su uso requiere solo una solicitud HTTP. Sin embargo, aunque Energy Meter no permite tomar mediciones del consumo de la GPU ni distingue el consumo por proceso, es una herramienta ideal para servir como base comparativa frente

a herramientas como Kepler 2.3 y Scaphandre 2.3, en el contexto del consumo energetico total del sistema.

Monitor

Energy Meter es una de las herramientas proporcionadas por el tutor del Tesis Final de Master Juan Gutierrez Aguado; está diseñada como alternativa a las herramientas Kepler 2.3 y Scaphandre, superando sus principales limitaciones. Permite adquirir métricas con granularidad muy baja, tanto del sistema completo como de procesos múltiples ejecutados en contenedores, y brinda información adicional sobre la carga de trabajo, su duración exacta y el procesamiento de datos. Además, ofrece la posibilidad de medir el consumo energético de la tarjeta gráfica mediante Nvidia SMI.

La herramienta se despliega mediante un archivo Docker Compose que incluye el contenedor de la aplicación y una base de datos MongoDB para almacenar los resultados de forma persistente, lo cual facilita su análisis posterior. Su despliegue es ligero y puede instalarse en cualquier entorno, independientemente del número de nodos, cumpliendo con las expectativas de las herramientas deseadas y resolviendo los puntos débiles de sus versiones anteriores.

Capítulo 3

Desarrollo de Proyecto

A continuacion se desarrollan los diferentes experimentos llevados a cabo, con la intencion de lograr cumplir los objetivos definidos en la fase de la introduccion [1](#).

3.1. Configuración de Entorno & Herramientas

3.1.1. Seleccion e Intalacion del Sistema Operativo

En un principio se decidió instalar el sistema operativo XUbuntu 26.04 en base a la recomendación de mi tutor de TFM, Juan Gutiérrez Aguado, ya que es un sistema operativo que él utiliza diariamente y sobre el que tiene una vasta experiencia de uso. Esto proporciona una capa extra de seguridad y de conocimiento sobre el entorno en el que se desarrollarían las pruebas, además del resto de recursos de conocimiento oficiales y no oficiales que existen sobre este sistema operativo.

Una vez finalizada la instalación, se hizo uso del gestor de versiones de drivers interno de Ubuntu, llamado 'ubuntu-drivers', para instalar la versión de los drivers de NVIDIA más compatible con la tarjeta gráfica NVIDIA Quadro M4000. Al comprobar si se podía acceder a los datos internos de consumo mediante `nvidia-smi`, se obtuvo un error que indicaba que no era capaz de comunicarse con la API interna de la tarjeta gráfica. Al revisar los mensajes proporcionados por el kernel, se pudo determinar que se trataba de un problema de compatibilidad de los drivers con la versión del kernel. Por este motivo se decidió instalar la versión 24.04 de XUbuntu [\[15\]](#), la cual hace uso de la versión 6.14 del kernel, solventando el problema de compatibilidad de los drivers.

Una vez reinstalado el sistema operativo, se procedió a reinstalar nuevamente los drivers de NVIDIA junto con los paquetes requeridos por `nvidia-smi`, de la misma forma que con la versión anterior. De nuevo, al tratar de acceder a los datos de consumo mediante 'nvidia-smi', apareció el mismo problema, pero al revisar los mensajes del kernel, se indicaba que el error no era de compatibilidad, sino la ausencia de un componente concreto llamado 'nvidia-drm'. Este componente forma parte del conjunto de módulos del kernel de NVIDIA que permite al kernel comunicarse con la tarjeta gráfica de NVIDIA [\[16\]](#). Al tratar varias veces de reconstruir el módulo manualmente, sin lograr éxito, se optó por buscar información en foros oficiales de NVIDIA y foros de la comunidad Ubuntu y Xubuntu, llegando a la conclusión de que era un problema que sucedía en algunas versiones de XUbuntu debido a dos razones principales: La versión Nouveau de los drivers ,y la

incompleta compatibilidad de los drivers con XUbuntu.

El primer problema estaba relacionado con el controlador libre Nouveau, desarrollado por la comunidad mediante técnicas de ingeniería inversa. Aunque este controlador proporciona compatibilidad básica con un amplio número de tarjetas gráficas NVIDIA, no implementa completamente las funcionalidades propietarias ofrecidas por los controladores oficiales, entre ellas la interfaz NVML, utilizada por herramientas como nvidia-smi para acceder a métricas avanzadas de la GPU, incluido el consumo energético. Para solucionar este problema fue necesario deshabilitar la carga del módulo Nouveau y reinstalar exclusivamente los controladores oficiales proporcionados por NVIDIA.

El segundo problema estaba relacionado con el mecanismo de instalación automática de controladores proporcionado por la utilidad ubuntu-drivers. Tras consultar la documentación oficial y diferentes foros de las comunidades de Ubuntu y Xubuntu, se comprobó que esta herramienta puede instalar automáticamente la versión más reciente del controlador junto con paquetes pertenecientes a versiones diferentes o dejar dependencias de instalaciones anteriores. Esta situación puede provocar inconsistencias entre los módulos del kernel y las bibliotecas de usuario, impidiendo que componentes como nvidia-drm o nvidia-smi funcionen correctamente. Como consecuencia, fue necesario eliminar completamente todos los paquetes relacionados con NVIDIA y realizar una instalación manual de una versión del controlador compatible con la tarjeta gráfica y con el sistema operativo empleado.

Pese a todo ello, no se logró una comunicación adecuada con la API interna de la tarjeta gráfica, lo que impidió el acceso a las métricas de consumo energético mediante nvidia-smi. Es por ello que se optó por instalar otra distribución de Linux más flexible, con mayor compatibilidad con los drivers de NVIDIA. Finalmente, se decidió instalar Debian, ya que cumplía con los requerimientos necesarios, además de ser una distribución que a lo largo de los años personalmente había utilizado para diferentes proyectos; por tanto, tenía experiencia de uso y conocía su funcionamiento interno y su filosofía de trabajo.

Se seleccionó la versión Debian 13 (Trixie), que en el momento de realizar este trabajo se encontraba dos versiones por detrás de la versión estable más reciente (Debian 15 - Duke) [17], siguiendo las recomendaciones de la comunidad de Debian. Durante la instalación del sistema se habilitó la detección automática de la tarjeta gráfica y la instalación de los controladores propietarios de NVIDIA. Sin embargo, al intentar utilizar la GPU desde un contenedor Docker para ejecutar procesos de codificación acelerados mediante hardware, se obtuvo el error de compatibilidad mostrado en la Figura 3.1. Dicho error indicaba que la versión instalada del controlador no cumplía con la versión mínima requerida por el entorno de ejecución, siendo necesaria la versión 580 o superior. Tras actualizar los controladores a la versión requerida manualmente, fue posible acceder correctamente a la GPU desde los contenedores y ejecutar procesos de codificación mediante aceleración hardware, resolviendo así los problemas de compatibilidad entre los controladores de NVIDIA, Docker y las bibliotecas empleadas por FFMpeg.

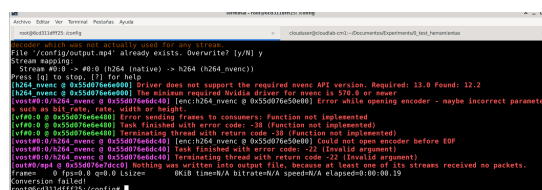


Figura 3.1: Error de compatibilidad con version de drivers de NVIDIA

3.1.2. Configuración e Instalación de Herramientas

A continuacio se exponen los procesos de instalacion, configuracion y modificaciones que se han llevado a cabo sobre las herramientas de toma de metricas. Todas esta herramientas monitorizan el consumo energetico del sistema y delas cargas de trabajo que desplegaran sobre contenedores basados en la imagen de ffmpeg gestionada por el grupo linux-server [18] [19].

Kepler

La instalación de Kepler en sistemas operativos Linux es relativamente sencilla, ya que la herramienta se distribuye en forma de ejecutable. Una vez descargado, dicho ejecutable debe añadirse a la variable de entorno *PATH* del sistema, permitiendo invocar la herramienta desde cualquier ubicación sin necesidad de indicar su ruta completa.

Una vez configurado el entorno, Kepler debe ejecutarse con privilegios de superusuario mediante el comando `sudo`, ya que necesita acceder a la interfaz *Running Average Power Limit* (RAPL) para recopilar las métricas de consumo energético, tal y como se muestra en el ejemplo de la Sección 3.1. La herramienta admite múltiples opciones de configuración, especificadas mediante *flags* durante su ejecución. Para la fase experimental desarrollada en este trabajo se han utilizado las siguientes:

- `-monitor.interval=1s`: establece un intervalo de muestreo de un segundo.
- `-metrics=container`: habilita la exportación de las métricas de consumo correspondientes a los contenedores monitorizados.
- `-metrics=node`: habilita la exportación de las métricas de consumo del nodo sobre el que se ejecuta la herramienta.
- `-experimental.gpu.enabled`: activa la recopilación de métricas asociadas a la GPU.

Por último, la salida estándar se redirige a `/dev/null`, ya que Kepler genera una gran cantidad de información relacionada con su configuración y estado de ejecución. Aunque esta información resulta útil durante las tareas de depuración, no es necesaria durante la ejecución de los experimentos y dificulta la interpretación de la salida por consola.

La obtención de las métricas se realiza mediante una petición HTTP utilizando la herramienta `curl` al puerto 28282, redirigiendo la respuesta a un archivo de salida para su posterior procesamiento. Una vez finalizada la recopilación de datos, Kepler debe detenerse manualmente, ya que, en caso contrario, continuará ejecutándose en segundo plano y seguirá recopilando métricas a la espera de nuevas peticiones, tal y como se muestra en la Sección 3.1.

Al tomar las primeras métricas con Kepler, se obtuvo un error en el apartado de consumo energético de la GPU, ya que marcaba como 0 el consumo energético cuando en realidad se había llevado a cabo una tarea de codificación. Al investigar los logs de salida de la aplicación, se observó un error al invocar una función interna llamada `'GetTotalEnergyConsumption'`, la cual pertenece a la librería `'libnvida-ml'` de `nvidia-smi`. Esta librería tiene la función de obtener el consumo de la tarjeta gráfica en cada instante que se hace

llamar a la función, y al parecer esta tenía problemas de compatibilidad con los drivers de NVIDIA que actualmente estaban instalados.

Tras investigar y comentar las posibles soluciones con mi tutor Juan, se decidió que se realizaría un hotfix sobre la herramienta Kepler, ya que se había logrado una estabilidad con los drivers y la reinstalación comprometería dicha estabilidad, además de no asegurar solventar el problema. Para ello se modificó aquellas funciones que llamaban a la función 'GetTotalEnergyConsumption' para que retornaran siempre 0 en su lugar. Una vez finalizado la modificación sobre el código de Kepler, se recompiló el código sobre un contenedor de Docker definido a partir de un Dockerfile ??.

Para comprobar que el hotfix ha funcionado adecuadamente, se lanzó una tarea de codificación y se monitorizó el consumo energético, tanto con Kepler como con nvidia-smi. Tras analizar y procesar dando como resultado unos resultados muy cercanos, los cuales tenían una desviación entre sí del 1,07%. Es necesario remarcar, que Kepler está devolviendo la potencia de la GPU durante el proceso de codificación, lo que implica que para obtener el consumo en Julius es necesario multiplicarlo por la duración del proceso de codificación.

```
1  #Toma de Metricas con Kepler
2  sudo ./kepler --metrics=container --monitor.interval=1s --
3  metrics=node --experimental.gpu.enabled > /dev/null &)
4  wait_finish_container() #ESPERA FINALIZACION PROCESO DE
5  CODIFICACION
6
7  #Recoleccion de metricas y finalizacion de la herramienta
8  curl -s http://localhost:28282/metrics > "nombre_archivo.metrics"
9
10 sudo pkill kepler 2>&1 &
```

Listing 3.1: Ejemplo de Ejecución y Recolección de métricas de la herramienta Kepler

Scaphandre

Scaphandre puede ser desplegado de varias formas, pero la más sencilla es desplegarlo empleando el contenedor de Docker con la instrucción 3.2. Esta instrucción ha sido modificada para desplegar Scaphandre en un estado de 'sleep', a la espera de instrucciones, ya que únicamente queremos tomar métricas cuando el proceso de codificación inicie y no queremos que el arranque del contenedor añada ruido a las métricas de consumo.

```
1  docker run --name scaphandre -v /sys/class/powercap:/sys/class/
2  powercap -v /proc:/proc --privileged -d --entrypoint "sleep"
   hubblo/scaphandre:1.0.2 3600
```

Listing 3.2: Arranque y puesta a punto de Scaphandre

Para llevar a cabo la toma de métricas con Scaphandre, es necesario especificar cada cuánto tiempo se llevará a cabo la toma de métricas y durante cuánto tiempo, ya que, a diferencia de Kepler, Scaphandre sí que define un tiempo máximo de recolección de métricas. Además, también es necesario indicarle las métricas que se desean obtener y sobre

qué archivo debe hacer el volcado de dichas métricas. Como se observa en el fragmento de código 3.3 correspondiente a la fase de experimentación 1 3.2.4, las órdenes de inicio de toma de métricas se llevan a cabo a través de la instrucción 'docker exec', indicando los parámetros de duración, tiempo de métricas y métricas que se desean obtener. En nuestro caso concreto, le indicamos que queremos obtener todas las métricas y que estas deben ser volcadas al archivo `out.metrics`'. Una vez finalizado el proceso de codificación, detenemos el proceso de forma forzosa y extraemos el archivo resultante del contenedor, para acto seguido apagarlo.

Al desplegar `scaphandre` de esta forma, tenemos control sobre el momento exacto del inicio de la recolección de toma de métricas y cuándo finaliza dicho proceso, eliminando posible ruido derivado del arranque del contenedor y los subprocesos asociados, logrando una replicabilidad en cada iteración para la fase experimental.

```

1  delta_t=1 # Tiempo de toma de metricas
2  t_metrics=60 #Tiempo de
3  # Start monitoring with scaphandre
4  docker exec -d scaphandre sh -c "scaphandre stdout -s ${delta_t}
   } -t 60 --raw-metrics > out.metrics"
5
6  monitorizacion_contenedor() # Monitorizacion Proceso de
   transcodificacion en el contenedor
7  # Run process
8  # Stop scaphandre monitoring
9  docker exec scaphandre sh -c "pkill -f scaphandre"
10 scapfile=local_file_with_scaphandre_metrics.out
11 # Get data from scaphandre
12 docker cp scaphandre:/app/out.metrics ${scapfile}
13 # Change owner of file
14 sudo chown clouduser:clouduser ${scapFile}
15 # Stop scaphandre container
16 docker stop scaphandre
17

```

Listing 3.3: Ejemplo de Inicio y Recoleccion de Metricas con Scaphandre

Energy Meter

Para instalar Energy Meter unicamente es necesario desplegar el docker compose creado por Juan Guatierrez A.1, el cual se descarga del repositorio del proyecto la imagen `energy-monitoring:1.0` y escuchapeticiones en el puerto 6000.

Para tomar metricas del sistema con la herramienta una vez docker compose esta arrancado, unicamente es necesario realizar una peticion curl de la siguiente forma `curl -s http://localhost:6000/api/start`. Y para recoger los datos de consumo, se debe realizar una nueva peticion `curl -s http://localhost:28282/metrics > "output.json"` pero esta vez redirigiendo la salida a un fichero, ya que al detener la toma de una métrica esta nos devuelve los datos de consumo del sistema en formato json.

```

1 services:
2   global-monitor:
3     image: energy-monitoring:1.0
4     container_name: global-energy-mon

```

```
5     privileged: true
6     ports:
7       - "6000:6000"
8     volumes:
9       - /var/run/docker.sock:/var/run/docker.sock
10      - /sys/fs/cgroup/system.slice:/sys/fs/cgroup/system.slice
11      - ./docker/app.py:/app/app.py
```

Listing 3.4: Docker Compose que permite desplegar la herramienta Energy Meter

Monitor

Al igual que la anterior herramienta, Monitor ha sido desarrollada por Juan Gutierrez Aguado como alternativa a Kepler y Schaphandre. Para instalarla unicamente es necesario descargar el contenedor `*energy-monitoring:1.0*` del repositorio del proyecto de investigacion y desplegarlo utilizado docker compose [A.2](#). Este archivo compose expone la herramienta en el puerto 5000 para recibir peticiones de monitorizacion.

El fragmento de código mostrado en la Sección [3.5](#) presenta un ejemplo del proceso de inicio y finalización de la monitorización, así como de la posterior obtención de las métricas recopiladas. Como puede observarse, tanto el inicio como la finalización de la monitorización se realizan mediante peticiones HTTP enviadas con la herramienta curl al puerto 5000, donde se encuentra desplegada la API de la herramienta.

Antes de iniciar el proceso de monitorización es necesario construir un payload en formato JSON, que será enviado en el cuerpo de la petición HTTP. Este payload contiene la información necesaria para identificar y configurar la carga de trabajo que se desea monitorizar.

En primer lugar, el atributo `workload_id` identifica de forma única el proceso de codificación, permitiendo asociar todas las métricas recopiladas a una misma carga de trabajo. A continuación, el atributo `container` especifica el nombre del contenedor cuya ejecución será monitorizada. Este parámetro resulta especialmente importante, ya que la herramienta permite monitorizar múltiples contenedores de forma simultánea.

Otro de los parámetros obligatorios es `gpu_indices`, mediante el cual se indica la GPU sobre la que se desea realizar la monitorización. Gracias a este mecanismo es posible trabajar con sistemas que disponen de varias tarjetas gráficas, asignando diferentes cargas de trabajo a cada una de ellas y obteniendo métricas independientes para cada contenedor y para cada GPU utilizada.

Finalmente, el atributo `persist_data` permite indicar si las métricas recopiladas deben almacenarse de forma persistente en la base de datos MongoDB. Este parámetro admite dos valores: 0, para deshabilitar el almacenamiento persistente, y 1, para guardar los datos una vez finalizada la monitorización.

La recolección de las métricas puede hacerse de dos formas diferentes; La primera es a través de la respuesta que devuelve el comando que detiene la monitorización, el cual consisten en un json con el consumo energetico de varias partes del sistema así como de la tarjeta grafica; Y a través de una petición a `*id_proceso/samples*`, la cual devuelve cada una de las métricas obtenida en la primera opción pero en formato csv en el que cada línea corresponde a un instante de medición.

Cabe destacar que, con el objetivo de garantizar la obtención de al menos diez muestras durante los procesos de transcodificación tanto en CPU como en GPU, fue necesario modificar el parámetro `*SAMPLES_PER_SECOND*` definido en el archivo Docker Compose de la herramienta, incrementando su valor de 1 a 3. De este modo, la herramienta pasa a realizar tres muestreos por segundo de monitorización, lo que permite obtener un número suficiente de muestras incluso en aquellos procesos de transcodificación cuya duración es reducida

```

1      uid=$(uuid)
2      payload='{ "workload_id": "'$uid"',
3      "container": "'$container'",
4      "gpu_indices": [0],
5      "persist_data": 0}'
6      response=$(curl -s -H "Content-Type: application/json" -X POST
7      -d "$payload" http://localhost:5000/api/monitor)
8      id=$(echo "$response" | jq -r '.id')
9      echo "Monitoring container $id"
10
11     # Almacenamiento Resultados
12     output=$(curl -s -X DELETE http://localhost:5000/api/monitor\?
13     id\=$id)
14     echo "Command: $workload_command" > ${infoFile}
15     docker exec $container sh -c "ls -al /config/output.mp4" >> ${
16     infoFile}
17     echo "Output: $output" >> ${infoFile}
18     docker exec $container sh -c "ffprobe -v error -select_streams
19     v -of default=noprint_wrappers=1:nokey=1 -show_entries stream=
20     r_frame_rate,width,height,duration,bit_rate -of default=
21     noprint_wrappers=1 /config/output.mp4" >> ${infoFile}
22
23     curl -s http://localhost:5000/api/monitor/workload/$uid/samples
24     > ${csvFile}

```

Listing 3.5: Código que ejemplifica el proceso de inicio y recolección final de métricas de consumo con la herramienta Monitor

3.1.3. Estado Final del Sistema

Una vez finalizado el conjunto de ajustes y de herramientas necesarias para llevar a cabo las diferentes fases experimentales, se han tomado métricas de consumo del sistema. El consumo energético del sistema, teniendo arrancada la herramienta EM y el contenedor de codificación (en un estado de sleep), se obtiene que el sistema está consumiendo de forma pasiva unos 544-545 julios. Por otra parte, la tarjeta gráfica está consumiendo unos 11 vatios de forma pasiva.

3.2. Selección de Herramienta para la toma de Métricas de Consumo

En esta primera fase de experimentación se han llevado a cabo una comparativa entre las diferentes herramientas disponibles para la toma de métricas de consumo sobre cargas de trabajo desplegadas en contenedores. Estas cargas de trabajo están basadas en la codificación de video y se llevarán a cabo empleando la herramienta **ffmpeg**.

Se ha utilizado un video para esta fase experimental, ya que el objetivo principal es determinar qué herramienta es capaz de tomar el conjunto de métricas de consumo más precisas. Este video consta de una duración de 50 segundos con un CRF 0 a una calidad 4K y con un bitrate de 1012693 kbps (1012,693 Mbps). Pese a que no es un video con una gran duración, tiene una gran calidad con 0 pérdida y un bitrate muy elevado, proporcionando suficiente carga de trabajo al codificador. Permitiendo extraer suficientes métricas en un tiempo prolongado del proceso de codificación mediante CPU y GPU, ya que, por lo general, los códecs que utilizan la GPU para llevar a cabo la codificación son notablemente más rápidos que los codificadores que utilizan codificación únicamente sobre CPU.

3.2.1. Desarrollo de los experimentos de la primera fase

En total se han definido cuatro escenarios experimentales, cuyo objetivo es evaluar la capacidad de cada herramienta para recopilar métricas de consumo energético, así como el nivel de granularidad de la información obtenida. Para la ejecución de estos experimentos se ha desarrollado un conjunto de scripts en Bash, encargados de automatizar tanto el despliegue de las pruebas como la recopilación de las métricas generadas por cada herramienta. Cada escenario se ejecutará un total de diez veces y, entre conjunto de iteraciones consecutivas, se establecerá un intervalo mínimo de dos minutos. Este intervalo tiene como objetivo reducir la influencia que el estado térmico del sistema tras las ejecuciones anteriores pueda ejercer sobre las mediciones, así como minimizar las variaciones inherentes a la ejecución de las cargas de trabajo. De este modo, se pretende obtener resultados más representativos y consistentes.

Los scripts empleados para realizar las pruebas con Kepler [A.3](#) y Scaphandre [A.4](#) comparten una estructura común, adaptando únicamente el proceso de inicialización y la recopilación de métricas a las particularidades de cada herramienta. En ambos casos es necesario especificar la ruta del vídeo que será codificado, el nombre del contenedor encargado de ejecutar el proceso de codificación y el directorio en el que se almacenarán las métricas obtenidas.

Por otra parte, los scripts desarrollados para la herramienta Monitor [A.5](#) y [A.6](#) requieren una configuración ligeramente más detallada. Además de indicar el contenedor que ejecutará la codificación y el directorio de salida de los resultados, es necesario definir el nombre del experimento, el número de iteraciones que se realizarán y el nombre de los archivos en los que se almacenarán las métricas recopiladas durante cada ejecución.

Cabe destacar que, en el caso de monitor, se han empleado dos scripts con el fin de evitar perder más tiempo y poder proseguir con la fase experimental 2, y poder cumplir con los plazos de entrega. Ambos scripts son prácticamente el mismo, a diferencia de [A.6](#) el cual lleva a cabo las acciones del script [A.5](#) dos veces, almacenando los datos resultantes de la recolección en archivos diferenciados.

3.2.2. Post-procesamiento de las Métricas de Consumo Energético

Una vez finalizada la fase de experimentación, el procesamiento de los datos obtenidos en cada iteración se llevará a cabo mediante un nuevo conjunto de scripts desarrollados en Bash. Estos scripts se encargan de procesar las métricas recopiladas en cada ejecución, calcular el valor medio para cada escenario experimental y almacenar los resultados en un archivo CSV.

En el caso de Kepler [A.7](#) y Scaphandre [A.8](#), el archivo CSV se genera en el mismo directorio donde se encuentran almacenados los datos de cada iteración. Por el contrario, la herramienta Monitor almacena el archivo resultante en un directorio denominado `data`. En todos los casos es necesario indicar la ruta donde se encuentran los datos que se desean procesar, así como el nombre del archivo CSV de salida.

Aunque los archivos CSV generados por cada herramienta presentan una nomenclatura diferente, todos ellos contienen información equivalente relativa a las métricas de consumo energético recopiladas durante la experimentación. En las siguientes figuras se muestra un ejemplo de la cabecera generada por cada herramienta:

- Kepler [A.1](#)
- Scaphandre [A.2](#)
- Monitor [A.3](#)

Finalmente, para generar las tablas y gráficas comparativas entre las distintas herramientas en cada escenario experimental, se ha desarrollado un script en Python [A.10](#). Este script procesa los archivos CSV generados por cada herramienta, calcula las métricas agregadas necesarias para el análisis comparativo y almacena los resultados en un nuevo archivo CSV. Dicho archivo constituye la entrada para la generación de las gráficas de barras histográficas y tablas comparativas que se presentan en el capítulo de resultados.

A continuación se detalla una lista de los posibles valores y su representación en la siguiente lista:

- `em_total_energy`: Energía total del sistema estimada por Energy Meter en base a las métricas recolectadas de RAPL
- `node_total_joules`: Energía total del sistema estimada por la herramienta
- `processN_energy_joules`: Energía total consumida por el proceso de transcodificación N estimado por la herramienta
- `gpu_node_joules`: Energía consumida por la GPU durante el proceso de codificación estimado por la herramienta

3.2.3. Análisis de los resultados obtenido en cada uno de los escenarios

Escenario 1: Transcodificación sobre un contenedor empleando CPU y GPU

En este primer escenario se le da acceso al contenedor tanto a la GPU como al número total de cores de la CPU. Con esta comparativa se espera poder medir las capacidades

basicas de cada una de las herramientas al tratar de determinar el consumo del nodo y de la gpu, asi como del proceso que se estan ejecutando sobre este en comparacion a las metricas de energia recolectadas por la herramienta externa Energy Meter.

Para llevar a cabo esta fase experimenta se le ha indicado a docker que haga uso del runtime de nvidia(`--runtime=nvidia`), el cual dota al contenedor de acceso a la GPU, y posteriormente se define un codificacion compatible con la aceleracion mediante GPU NVIDIA (`h264_nvenc` o `hevc_nvenc`). Como los datos resultantes se ha obtenido la siguiente grafica comparativa 3.2.

Cabe destacar que la energía total medida por Energy Meter (`em_total_energy`) y por las herramientas de métricas respecto al nodo (`node_total_joules`) miden el consumo energético basándose en RAPL, lo cual implica que ninguna de estas medidas incluye el consumo de la tarjeta gráfica. Por tanto, para conocer el consumo total del sistema durante el proceso de codificación empleando tarjeta gráfica, se deben sumar las métricas de consumo del nodo y la métrica de consumo de la GPU (`gpu_node_joules`) `gpu_node_joules + em_total_energy = Total_Energy_System`.

En cuanto a la capacidad de tomar métricas de las herramientas, se observa que Scaphandre es la única herramienta que no ha sido capaz de tomar métricas de consumo energético de la GPU. En cambio, tanto Kepler como Monitor sí que han sido capaces de medir el consumo energético, siendo en ambos casos bastante similares. Esto nos sirve de indicativo de que el hotfix realizado sobre Kepler se comporta segun la referéncia de consumo esperada, ademas de indicar que la capacidad de tomar métricas de consumo de la tarjeta gráfica de ambas herramientas parece ser adecuada.

Si nos centramos en la capacidad para medir el consumo energético del nodo, Monitor es la que menor discrepancia presenta respecto a la energía que Energy Meter ha tomado del sistema, con una tasa de error del 0,4 % calculada empleando la formula 3.2.3. Seguida por Scaphandre quien presenta una tasa de error del 3,6 %, y , en ultimo lugar, Kepler quien presenta un tasa de error del 7,5 %.

$$E_{ErrTotal} = \frac{|E_{metrica_herramienta_nodo} - E_{metrica_energy_meter}|}{E_{metrica_energy_meter}} * 100$$

En cuanto a la energia que el proceso de transcodificacion ha consumido, con Monitor y con Scaphandre se observa que existe una diferencia entre el consumo energético del nodo(`node_total_joules`) y el consumo del proceso(`node_total_joules`) llevado a cabo en el contenedor. Esta diferencia ronda los **100-200 Julios**, que supone entre un 5,8 % y 8,9 % respecto al total de la energia consumida por el nodo. Este hecho puede deberse a la existencia de ruido energetico posiblemente deribado de aquellos componentes hardward sobre los que RAPL no puede tomar metricas. De ahora en adelante nos referiremos a esta energia no medible como *ruido del sistema* $E_{ruido_sistema}$.

Cabe destacar, que en el caso de las metricas de Kepler, el consumo energético del proceso presenta muy poca diferencia respecto al consumo energetico del nodo, siendo la diferencia energetica mucho mayor respecto a la energia medida por Energy Meter (`em_total_energy`). Este hecho puede deberse a que hipeteticamente Kepler este calculando la energia total del nodo en base a la energia del proceso de la siguiente forma $E_{energi_nodo} \approx E_{ruido_sistema} + \sum_0^n E_{proceso_n}$.

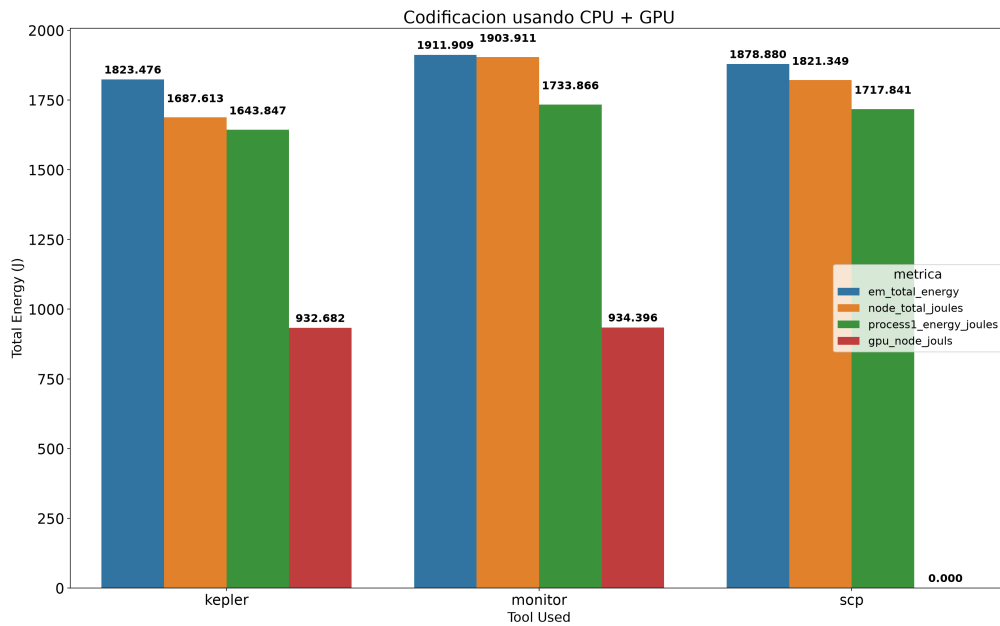


Figura 3.2: Histograma resultante de las metricas de consumo energetico en julios sobre una tarea de codificación empleando GPU y CPU

Escenario 2: Transcodificación empleando un contenedor unicamente con CPU

En el siguiente escenario, se ha llevado a cabo la transcodificación sobre un contenedor empleando un encoder que utiliza únicamente CPU. Los resultados obtenidos han dado como resultado la siguiente gráfica de histogramas 3.3.

En relación a las métricas de consumo del nodo, se aprecia nuevamente que Monitor tiene una menor diferencia respecto a al consumo energético del nodo frente al consumo energético medido por Energy Meter. Al igual que en el anterior escenario, se observa que existe una diferencia entre el consumo energético del proceso de codificación y el consumo energético total del nodo, y que de la misma forma el consumo energético del proceso es anormalmente bajo. Unos resultados muy similares a los del escenario 1, sin el consumo energético de la GPU.

Escenario 3: Transcodificación empleando un contenedor limitado a tres cores

En este tercer escenario, se ha limitado el numero de cores a los que el contenedor tiene acceso con el fin de medir la granularidad de cada una de las herramientas en relacion a las metricas de consumo. Para limitar el numero de cores se han añadido las opciones:

- `--cpus="3"`: Numero de cores que el contenedor puede utilizar.
- `--cpuset-cpus=1-3`: Rango de las cores de las que dispone, en este caso reservamos la 0 para el sistema.

Analizando el consumo energetico general de las diferentes metricas, se observa un aumento del consumo energetico del 40% respecto al escenario que no presentaba limitacion en el numero de cores de la CPU. En cuanto a la metrica de consumo energetico

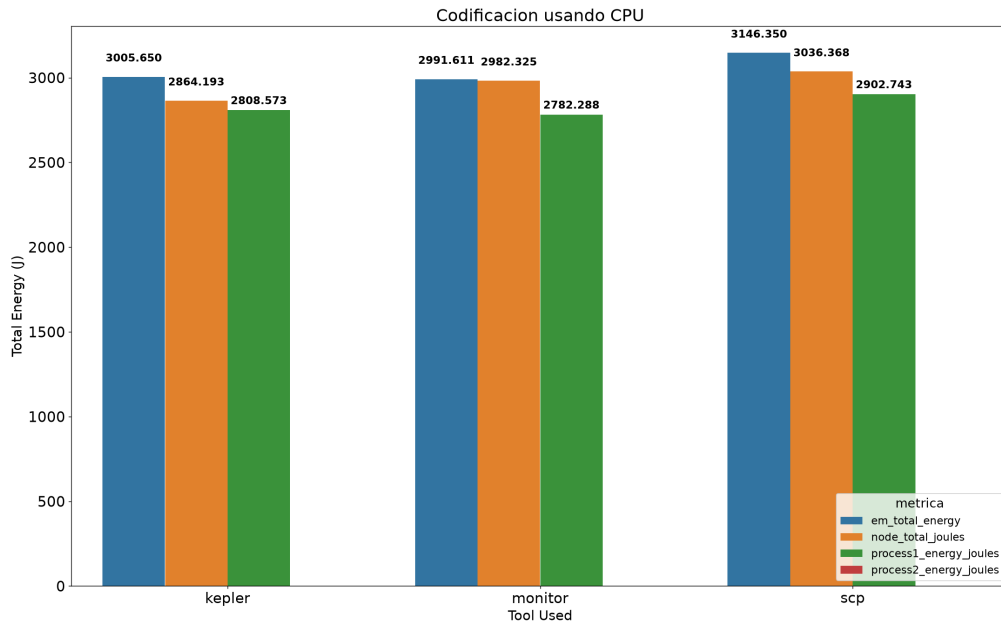


Figura 3.3: Histograma comparativo de las metricas de consumo energetico en julios de las tres herramientas en una tarea de codificación empleando exclusivamente la CPU

del nodo, Monitor es el que menor discrepancia tiene respecto a la energía medida por Energy Meter con una tasa de error del 0,25 %. Seguido por Scaphandre, quien presenta un tasa de error del 2,59 % respecto a la energía medida por Energy Meter, y en ultimo lugar, Kepler el cual presenta un tasa de error del 62,59 %, una gran diferencia respecto tasa de error de las dos herramientas.

En cuanto a la metrica de energía en relacio al proceso, se ara uso de la siguiente ecuación 3.2.3 para calcular el porcentaje de energía respecto al total consumido por el nodo es atribuido al proceso de transcodificación. De las tres herramientas, Monitor es la menor diferencia presenta respecto a la energía total del nodo, con una diferencia de **753.76 julios** que supone un 82,10 % de la energía total. En el caso de Scaphandre se observa una diferencia de **1830.56 julios** respecto a la energía del nodo, lo que supone un 48,88 %. En cambio Kepler presenta una discrepancia minima respecto a la energía del nodo, sendo del **24.842 julios**(98,41 %), lo que refuerza la hipotesis de que Kepler esta ajustando la energía del nodo basando se en la ecuacion 3.2.3.

$$Porcentaje_Energia = \frac{E_{proces_codificacin}}{E_{total_nodo}} * 100$$

Adicionalmente, el conjunto de resultados obtenidor tambien refuerzan la hipotesis de que algunos componentes hardware del nodo, en el que se estan ejecutando las pruebas, estan consumiendo recursos del sistema pero aparentemente RAPL no es capaz de medirlos.

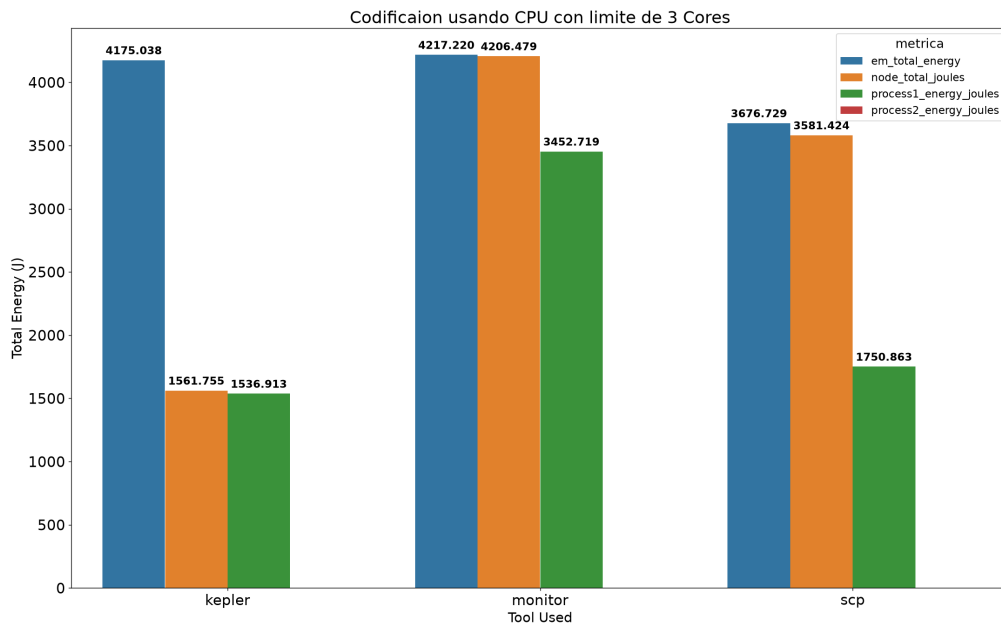


Figura 3.4: Histograma resultante delas metricas de consumo sobre una tarea de codifi-
cacion emplendo un contenedor limitado a 3 cores

Escenario 4: Transcodificacion concurrente empleando dos contenedores limi- tados a tres cores cada uno

Finalmente, en el cuarto escenario experimental se limita el acceso a los recursos disponibles, distribuyendo la carga de trabajo entre dos contenedores. A cada contenedor se le asignan tres núcleos de CPU, reservando nuevamente el núcleo 0 para las tareas propias del sistema operativo. De este modo, el primer contenedor utiliza los núcleos 1–3, mientras que el segundo utiliza los núcleos 4–6. Asimismo, se restringe el acceso a la GPU de forma que ambos contenedores puedan ejecutar simultáneamente sus respectivos procesos de transcodificación bajo las condiciones establecidas para el experimento.

En esta última fase se pretende evaluar la precisión y granularidad de las herramientas al monitorizar de forma simultánea el consumo energético asociado a dos contenedores que ejecutan procesos de transcodificación independientes. Los resultados obtenidos se muestran en la Figura 3.5.

En relación a la estimacion de consumo energético respecto a la energia del nodo, Monitor presenta los resultados más próximos a la referencia obtenida mediante Energy Meter, con una tasa de error del 7,23 %. Seguido de Scaphandre, cuya medición presenta una diferencia de aproximadamente **200-300 julios**, lo que supone una tasa de error del 29,62 %, respecto a dicha referencia. En el caso de Kepler, se observa una discrepancia considerable en la estimación del consumo energético del nodo, con una tasa de error del 1,32 %.

No obstante, la métrica obtenida presenta una magnitud similar a la suma del consumo atribuido a los dos contenedores monitorizados, reforzando la hipótesis planteada anteriormente, según la cual Kepler podría estar estimando el consumo del nodo a partir del consumo asociado a los procesos monitorizados, en lugar de proporcionar una medi-

ción independiente del consumo total del sistema.

Si se analiza específicamente el consumo energético atribuido a los procesos, Scaphandre proporciona una estimación total de **4653,025 julios**. Por otro lado, el consumo energético total del nodo medido por la misma herramienta asciende a **6610 julios**. Por tanto, el consumo atribuido a los procesos representa aproximadamente el 70,4 % del consumo total estimado para el nodo. Esta proporción resulta razonable, ya que una parte del consumo energético del nodo corresponde a otros componentes y procesos del sistema que no forman parte de las cargas de trabajo monitorizadas.

Por su parte, Monitor proporciona una estimación del consumo energético asociado a los procesos cuya suma asciende a **6212,860 julios**, frente a un consumo energético total del nodo de **6697,325 julios**. Esto supone una diferencia de **484,465 julios** y significa que Monitor atribuye aproximadamente el **92.77 %** del consumo energético total del nodo a los procesos ejecutados en los dos contenedores. Este resultado es coherente con las condiciones del escenario descrito anteriormente, en el que la mayor parte de la carga de trabajo del sistema corresponde a los procesos de transcodificación ejecutados simultáneamente en ambos contenedores.

En comparación con Scaphandre, que atribuye aproximadamente el **70.40 %** del consumo total del nodo a los procesos monitorizados, Monitor presenta una diferencia de **22.37 puntos porcentuales** en la proporción de consumo atribuida a las cargas de trabajo. Además, Monitor proporciona un mayor nivel de granularidad en la recopilación de las métricas, permitiendo obtener información más detallada sobre el consumo energético asociado a cada contenedor y a los procesos que se ejecutan en ellos.

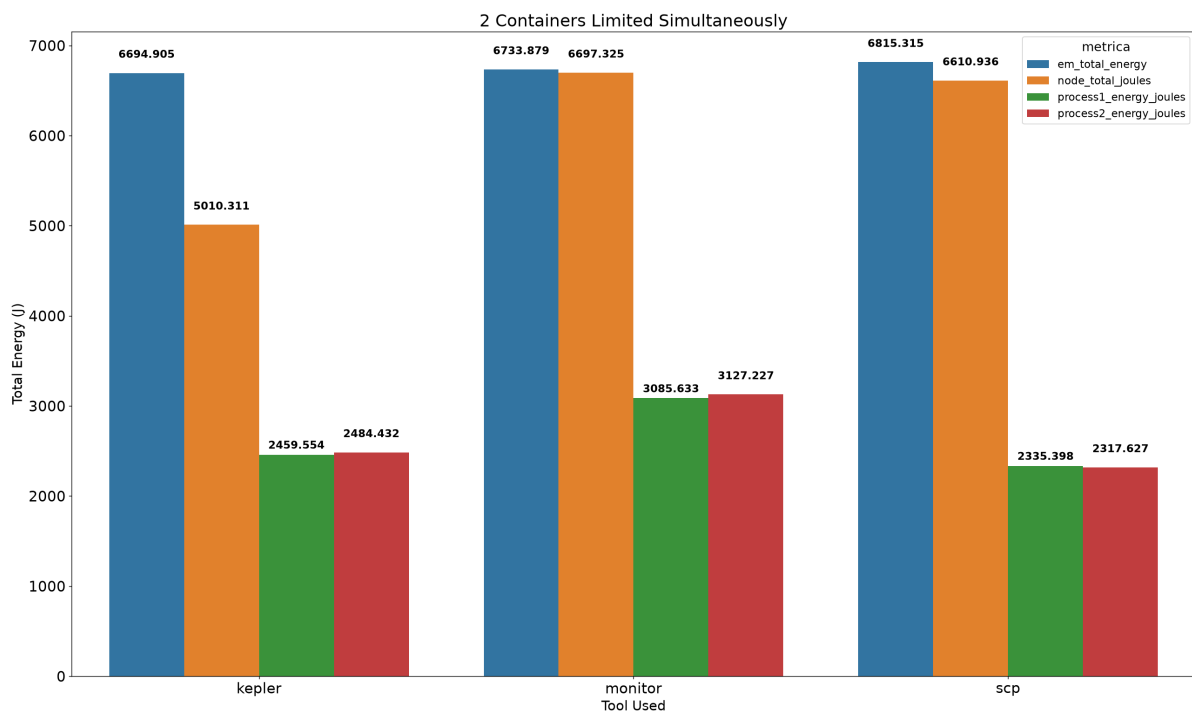


Figura 3.5: Histograma resultante de las metricas de consumo de dos contenedores limitados a 3 cores cada uno ejecutando tareas de codificacion simultaneamente

3.2.4. Conclusiones fase 1 de la experimentacion

Los experimentos llevados a cabo en esta primera fase han permitido comparar el comportamiento de *Scaphandre*, *Kepler* y *Monitor* bajo distintas condiciones de ejecución, teniendo en cuenta tanto el consumo energético total del nodo como la posibilidad de atribuir dicho consumo a los procesos monitorizados. Los resultados obtenidos muestran diferencias importantes entre las herramientas, especialmente en situaciones donde se limitan diferentes recursos del sistema y se varía la carga de trabajo e incluso el número de procesos que se ejecutan al mismo tiempo.

En cuanto a la estimación del consumo energético del nodo, *Monitor* es el que menor discrepancia respecto a la medida de referencia utilizada presenta, de forma general. En los escenarios estudiados, los errores obtenidos por esta herramienta se mantienen en valores bajos, llegando a un error del **0.25 %** en el escenario 3 y un error del **0.31 %** en el escenario 2. *Scaphandre* también da estimaciones cercanas a las de referencia en algunos casos, pero tiene mayor diferencia en el escenario 4. Por su parte, *Kepler* presenta un comportamiento más variable, alcanzando un error del **62.59 %** en el escenario 3.

Además de la estimación del consumo del nodo, la capacidad de desagregar el consumo entre las cargas de trabajo constituye un aspecto especialmente relevante para los objetivos de este trabajo. En este sentido, *Monitor* presenta una mayor capacidad de atribución del consumo energético a los procesos monitorizados. Esto resulta especialmente significativo en el escenario 4, donde consigue atribuir a los procesos el **92.77 %** del consumo total medido para el nodo, frente al **70.40 %** obtenido por *Scaphandre* y el **22.37 %** obtenido por *Kepler*. Estos resultados indican que *Monitor* proporciona una mayor cobertura del consumo energético cuando se analizan varias cargas de trabajo de forma concurrente.

Por otra parte, los resultados alcanzados posibilitan la identificación de ciertas limitaciones de las herramientas evaluadas. *Scaphandre* tiene ciertas limitaciones a la hora de obtener información energética relacionada con algunos recursos de hardware, como la GPU. También tiene limitación a la hora de tomar métricas sobre múltiples procesos sobre múltiples contenedores en simultáneo, ya que, como se puede observar en la tabla comparativa 3.2.4, el error respecto a la energía medida por EM escala conforme al número de procesos aumenta.

En el caso de *Kepler*, si bien no tiene problemas para medir el consumo sobre la GPU, presenta discrepancias importantes a la hora de estimar el consumo energético del nodo, con un gran nivel de error, lo que restringe su elección como herramienta de toma de métricas de uno o varios procesos del sistema. Este factor dificulta la posibilidad de saber si las métricas recolectadas del conjunto de procesos están sobreestimadas o, por el contrario, muy por debajo del consumo real.

Estas diferencias muestran que es importante seleccionar la herramienta no solo por su capacidad de obtener una medida energética, sino también por la granularidad y nivel de atribución que ofrece.

Por lo tanto, y teniendo en cuenta la discrepancia respecto a la referencia, la capacidad de atribución del consumo a los procesos y el comportamiento observado en los distintos escenarios, se selecciona *Monitor* como herramienta para las siguientes fases del estudio. Esta selección facilitará una mayor precisión en la medición del consumo energético de las cargas de trabajo implementadas, elemento clave para posteriormente examinar el impacto energético de los diferentes escenarios de transcodificación estudiados en este trabajo.

	E 1	E2	E3		E4	
Herramienta	E.N(%) ¹	E.N(%)	E.N(%)	EP(%)	E.N(%)	EP(%) ²
Scaphandre	3.6	3.49	2.59	48.88	29.62	70,40
Kepler	7.5	4.71	62.59	98.41	1.32	22.37
Monitor	0.4	0.31	0.25	82.10	7.23	92,77

Cuadro 3.1: Tabla comparativa de errores de cada una de las escenarios experimentales

3.3. Consumo energetico en relacion al SITI y el Numero de Threas

En esta fase de experimentación tiene dos objetivos principales; El primero es demostrar la relacion entre el consumo energético y las métricas Spatial Information(SI) y Temporal Information(TI) de los videos empleados en las pruebas; Y el segundo consiste en buscar una correlacion entre el consumo energetico y el numero the threads empleados en el proceso de codificacion limitando tambien el numero de cores, en base a la siguiente relacion $C_{numero_cpus} \approx T_{numero_threads}$.

Con el fin de poder exponer de forma clara los resultados y conclusiones de ambos objetivos, se ha dividido la redacción en subfases por objetivo. Cada una de estas subfases expondra como se han llevado a cadabo cada uno de los experimentos, el post-procesamiento de los datos, el analisis de las graficas obtenidas y las respectivas conclusiones.

En cuanto a los experimentos, se han empleado un total de quatro codificadores 3.3; dos de ellos basados en codificacion mediante CPU (libx264 y libx265); y otros dos que combinan la CPU junto con la GPU (h264_nvenc y hevc_nvenc).

Tambien se ha variado el bitrate en cada transcodificacion de cada video, y dependiendo de si se trataba de un codec de la generacion x264 o de la generacion x265, se ha empleando una lista de bitrates concreta. La seleccion de dichos rangos de bitrate, se ha hecho en base a las recomendacions oficiales [20], y posteriormete se ha aumentando el rango del bitrate maximo y minimo con el fin de remarcar diferencias facilmente apreciables en el resultado final de codificacion y, simultaneamente, medir el esfuerzo requerido por el codec conforme el bitrate aumenta. Los rango de bitrate empleados son:

- libx264 y H264_NVENC: 10,20,25,30,35,40,50
- libx265 y H265_NVENC: 10,15,20,25,30,35,40

Por tanto, se ha trasncodificado cada video empleando los codificadores y los bitrates correspondientes de la lista, midiendo cada proceso con la herramienta Monitor. Ya que como se ha demostrado en la seccion anterior 3.2.4, es la herramienta mas precisa y mas adecuada para obtener los datos de consumo, asi como aportar mayor granularidad sobre estas metricas.

En ambas fases se ejecutara unicamente un contenedor por prueba, sin concurrencia de procesos o contenedores, basados en la imagen **linuxserver/ffmpeg:8.0.1** pero con un sleep de 5 dias ya que las pruebas de este bloque pueden demorar varias horas en dar

¹EN: Error de la energia del nodo respecto a la energia total del sistema obtenida con Energy Meter

²EP: Energia del proceso respecto a la energia del nodo calculada por la herramienta

CODECS	CPU	GPU
libx264	X	-
libx265	X	-
h264_nvenc	X	X
hevc_nvenc	X	X

Cuadro 3.2: Codecs seleccionados para la fase de experimentacion

resultados. El resto de parametros de la declaración del contenedor son identicos en cada subfase, a diferencia de el nombre y del numero de CPU disponibles, ya que al contrario que en la fase 1 en la fase 2 si que se limitan el numero de contenedores de la misma forma que en la seccion 3.2.4.

```

1  #Sin limitaciones
2  docker run -d --name ffmpeg-encoder --network
   global_monitoring_default -v "$(pwd)/VIDEOS":/config --runtime=
   nvidia --entrypoint sleep linuxserver/ffmpeg:8.0.1 18000
3  #Limitao a X cores de Y a Z
4  docker run -d --name ffmpeg-encoder --network
   global_monitoring_default --cpus="X" --cpuset-cpus="Y-Z" -v "$(
   pwd)/VIDEOS":/config --runtime=nvidia --entrypoint sleep
   linuxserver/ffmpeg:8.0.1 18000

```

Listing 3.6: Declaracion de container basica para las pruebas de la subfase 1 y subfase 2

En cuanto a los video seleccionados para llevar a cabo los distinto experimentos, nuevamente, se ha empleado la base de video ya existente del proyecto ecostream. En concreto se han seleccionado los chunks que provienen de la tabla de videos 3.3, descartando aquellos chunks que contengan escenas de creditos y fundidos a negro, tanto al incio como al final. Cada uno de los chunks pertenecientes a los videos de la tabla, han sido codificados a 4k con un CRF 0 y con 30 fps utilizando el encoder libx264. Ademas tienen un perfil de color yuv420p(4:4:4) con una profundidad de 8 bits ?? y 48 frams por chunk A.1.2.

Nombre del Video	Duracion	Chunks Utilizables
AncientThought_CableLabs	3'	[0,75]
Eldorado_CableLabs	3'	[0,85]
IndoorSoccer_CableLabs	4'11"	[0,109]
LifeUntouched_CableLabs	3'20"	[0,139]
LiftingOff_CableLabs	3'28"	[0,91]
MomentofIntensity_CableLabs	3'10"	[0,83]
SecondsThatCount_CableLab	5'30"	[0,134]
Sintel_Bender	15'24"	[4,370]
Skateboarding_CableLabs	4'50"	[0, 125]
TearsOfSteel_Bender	12'20"	[4,290]
UnspokenFriend_CableLabs	4'20"	[0,113]

Cuadro 3.3: Lista de video de los cuales se han realizado la seleccion de chunks para el conjunto de pruebas experimentales

3.3.1. Clasificación y Selección de Chunks en base al SITI

El conjunto de videos que será utilizado para en el apartado del desarrollo 3.3 proviene de la base de videos que el proyecto EcoStrema tiene, y que proviene principalmente de [21]. Estos videos tiene la característica que estan codificados en formato H.265 sin perdida de calidad y sin compresion, ademas de

Selección de muestras Basado en SI y TI

Para la selección del conjunto de videos que serán utilizados durante la fase de experimentación 2 3.3, se espera poder comprobar si el contenido del video, y más concretamente, la cantidad de información que contiene, afecta a la cantidad de energía que consume el contenedor durante el proceso de codificación. Es por este motivo que se han empleado los parámetros Spatial Information (SI) y Temporal Information (TI) para la selección de chunks que compondrán los videos.

Estos parámetros de calidad subjetiva se encuentran definidos por el International Telecommunication Union Telecommunication Standardization Sector (ITU) [22], y se emplean de forma conjunta como métricas de calidad comprendida entre 0 y 100, donde Spatial Information (SI) representa la cantidad de detalles y objetos que aparecen en una escena, y el Temporal Information (TI) define la cantidad de cambios que suceden en una escena a lo largo del tiempo.

Para ellos, se han seleccionado los vídeos mostrados en la Tabla 3.3.1. A partir de cada uno de ellos se calcularán los índices de Información Espacial (SI) e Información Temporal (TI) de sus correspondientes chunks. Posteriormente, se generarán vídeos de 10 segundos de duración a partir de un único chunk, el cual se reproducirá de forma cíclica hasta completar el tiempo establecido. Este procedimiento permite mantener constantes los valores de SI y TI durante toda la secuencia, evitando que las variaciones del contenido afecten al análisis del consumo energético.

Video	Origen
Eldorado	Cable Labs
Skateboarding	Cable Labs
LifeUntouched	Cable Labs
LiftingOff	Cable Labs
Sintel	Bender
SecondsThatCount	Cable Labs
IndoorSoccer	Cable Labs

Cuadro 3.4: Videos Seleccionados para las pruebas

Con el objetivo de disponer de una muestra representativa de diferentes tipos de contenido, los chunks se han clasificado en cuatro categorías según sus valores de SI y TI, tal y como se muestra en la Lista 3.3.1. Para cada categoría se seleccionarán cinco chunks, obteniendo un total de veinte vídeos para la experimentación. Esta muestra permitirá analizar la posible relación entre las características espaciales y temporales del contenido y el consumo energético asociado al proceso de codificación.

- Alto TI y bajo SI (High TI – Low SI).
- Bajo TI y alto SI (Low TI – High SI).

- Bajo TI y bajo SI (Low TI – Low SI).
- Alto TI y alto SI (High TI – High SI).

Para lograr calcular y visualizar los valores de SI y TI correspondientes a cada chunk, se han desarrollado dos scripts en Python. El primero [A.1.2](#) emplea la biblioteca *siti-tools* [23], disponible en el gestor de paquetes de Python [PyPI](#), para calcular las métricas de Información Espacial (SI) e Información Temporal (TI) de cada chunk y almacenar los resultados en un archivo CSV.

El segundo script [A.1.2](#) utiliza dicho archivo como entrada para generar una gráfica de dispersión, en la que el eje X representa los valores de SI y el eje Y los valores de TI. Cada punto de la gráfica corresponde a un chunk, cuya posición viene determinada por las métricas calculadas previamente.

3.3.2. Consumo energetico en relacion al SITI

3.3.3. Relación entre numero de cores, tiempo de codificación y consumo energetico

Capítulo 4

Pruebas y Resultados

Capítulo 5

Conclusiones y Trabajo Futuro

Apéndice A

Apéndice

A.1. Subapendice: Experimentacion

Docker Compose de la herramienta Monitor

Imagen de Docker empleada para recompilar Kepler

```
1 # Build the binary
2 FROM golang:tip-trixie
3
4 # Install cross-compiler if building for a different architecture
5 RUN apt-get update && apt-get install -y --no-install-recommends \
6     gcc-x86-64-linux-gnu libc6-dev-amd64-cross && rm -rf /var/
7     lib/apt/lists/*;
8
9 WORKDIR /workspace
10 COPY . .
11 RUN CROSS_CC=x86_64-linux-gnu-gcc; \
12     make build \
13     CGO_ENABLED=1 \
14     PRODUCTION=1 \
15     GOARCH=amd64 \
16     ${CROSS_CC:+CC=${CROSS_CC}} \
```

Listing A.1: Imagen de contenedor en Go utilizada para recompilar la version modificada de Kepler

Configuracion del Sistema y Herramientas

```
1 services:
2   monitor:
3     image: routerdi1315.uv.es:33443/energy-monitoring:1.0
4     container_name: container-energy-mon
5     privileged: true
6     ports:
7       - "5000:5000"
8     volumes:
```

```

9      - /var/run/docker.sock:/var/run/docker.sock
10     - /sys/fs/cgroup/system.slice:/sys/fs/cgroup/system.slice
11     - ./data:/data
12     #- ./docker/app.py:/app/app.py
13     #- ./docker/monitor.py:/app/monitor.py
14     #- ./docker/mdb.py:/app/mdb.py
15     environment:
16     - MONGO_HOST=mongodb
17     - MONGO_PORT=27017
18     - NVIDIA_VISIBLE_DEVICES=all
19     - MONITOR_GPU=True
20     - SAMPLES_PER_SECOND=3
21     runtime: nvidia
22
23     mongodb:
24     image: mongo:8.2.2-noble
25     container_name: mongodb
26     volumes:
27     - ./mongodb/data:/data/db
28     ports:
29     - "27017:27017"
30     command:
31     - "--quiet"

```

Listing A.2: Docker Compose que permite desplegar Monitor

A.1.1. Experimentación Fase 1

```
IT,container,em_host,gpu_process_joules,core_process_joules,dram_process_joules,package_process_joules,total_process_joules,total_node_joules
```

Figura A.1: Header CSV del archivo de datos procesados de Kepler

```
IT,pid,em_host,total_energy_host,cpu_usage_percentage,total_process_joules
```

Figura A.2: Header CSV del archivo de datos procesados de Scaphandre

```
tool,case,em_host,gpu_process_joules,node_total_joules,process1_energy_joules
```

Figura A.3: Header CSV del archivo de datos procesados de Monitor

Arranque y toma de metricas con Kepler

```

1  #!/bin/bash
2
3  if [ $# -lt 1 ]; then
4      echo "Numero de argumento incorrectos :- ("
5      echo "***** Debes de indicar *****"
6      echo "**\n Numero_Iteracion      **"

```

```

7     exit 1
8 fi
9
10 #Funcion de stop until proceso finalice del todo
11 function function_Endding_Procces() {
12     local container=$1
13     echo "Codificacion en proceso en ${container}..."
14     sleep 1
15     while docker top ${container} | grep -q "ffmpeg"; do
16         sleep 0.5
17     done
18     sleep 1
19     echo "Codificacion Finalizada"
20 }
21
22
23 # ++++++ Toma de Medidas de Kepler ++++++
24 iteracion=$1
25 kind_iter="cpu"
26 dir="kepler/cpu" # Ubicacion del ARCHIVO final
27 fem="em/cpu/em_kepler_${kind_iter}_${iteracion}.json" # Ubicacion
   archivos finales em
28 kepler_raw_results="$dir/zdirty_raw_kepler_${kind_iter}_${iteracion}
   .metrics" # RAW Data
29 final_results_csv="$dir/kepler_10_it_${kind_iter}.csv" #Nombre del
   arcohivo final resultante
30 container=("ffmpeg-encoder")
31
32 for c in ${container[@]}; do
33     if ! docker ps --filter "name=${c}" --format "{{.Names}}" |
   grep -q "${c}"; then
34         echo " *** ERROR: Contenedor ${c} apagado ***"
35         exit 1
36     else
37         echo "Working ${c}"
38     fi
39 done
40
41 sudo kepler --metrics=container --monitor.interval=1s --metrics=
   node --experimental.gpu.enabled > /dev/null &
42 sleep 1
43
44 # Iniciar metricas con EM
45 curl -s http://localhost:6000/api/start
46 echo "EM start..."; echo ""
47 codec="libx264" #h264_nvenc libx264
48 for c in ${container[@]}; do
49     echo "${c} docker working"
50     docker exec ${c} ffmpeg -y -i /config/SkLab_3840x2160_crf0.mp4
   -c:v ${codec} -b:v 4M -vf scale=1280:720 -c:a copy /config/
   output.mp4 > /dev/null 2>&1 &
51 done

```

```

52 #Funcion de stop until proceso finalice del todo
53 for c in "${container[@]"; do
54     function_Endding_Procces ${c}
55 done
56
57 echo "EM stop...."; echo ""
58 curl -s http://localhost:6000/api/stop | jq . > "${fem}"
59
60 curl -s http://localhost:28282/metrics > "$kepler_raw_results"
61
62 sudo pkill kepler 2>&1 &
63 exit 0

```

Listing A.3: Script en bash empleado para llevar a cabo la toma de metricas de consumo energetico con la herramienta Kepler

Arranque y toma de metricas con Scaphandre

```

1  #!/bin/bash
2  #
3
4
5  if [ $# -lt 1 ]; then
6      echo "Numero de argumento incorrectos :- ("
7      echo "***** Debes de indicar *****"
8      echo "**\n Numero_Iteracion      **"
9      exit 1
10 fi
11
12 #Funcion de stop until proceso finalice del todo
13 function function_Endding_Procces() {
14     local container=$1
15     echo "Codificacion en proceso..."
16     sleep 1
17     while docker top ${container} | grep -q "ffmpeg"; do
18         sleep 0.5
19     done
20     sleep 1
21     echo "Codificacion Finalizada"
22 }
23
24 #Funcion para saber si el contenedor de scaphandre esta arrancado
25
26 function function_container_status() {
27     local container=$1
28     echo "Arrancando contenedor ${container}...."
29     while ! docker ps --format json | grep -q "${container}"; do
30         sleep 0.5
31     done
32     sleep 15
33     echo "Contenedor ${container} arrancado"
34

```

```

35 }
36
37
38 # ++++++ Toma de Medidas de scaphandre
39 # ++++++
40 iteracion=$1
41 fem="em/cpu/em_scp_cpu_${iteracion}.json"
42 dir="scp/cpu" # Ubicacion del ARCHIVO final
43 scapfile="$dir/zdirty_raw_scp_cpu_${iteracion}.metrics" # RAW Data
44 final_results_csv="$dir/scp_10_it_cpu.csv" #Nombre del arcohivo
45 # final resultante
46 container=("ffmpeg-encoder")
47 delta_t=1
48
49 for c in ${container[@]}; do
50     if ! docker ps --filter "name=${c}" --format "{{.Names}}" |
51     grep -q "${c}"; then
52         echo " *** ERROR: Contenedor ${c} apagado ***"
53         exit 1
54     fi
55 done
56
57 docker start scaphandre
58 function_container_status "scaphandre"
59
60 docker exec -d scaphandre sh -c "scaphandre stdout -s ${delta_t} -t
61     200 --raw-metrics > out.metrics"
62 sleep 1
63
64 curl -s http://localhost:6000/api/start
65 echo "EM start...."; echo ""
66
67 codec="libx264" #h264_nvenc libx264
68 for c in ${container[@]}; do
69     docker exec ${c} ffmpeg -y -i /config/SkLab_3840x2160_crf0.mp4
70     -c:v ${codec} -b:v 4M -vf scale=1280:720 -c:a copy /config/
71     output.mp4 > /dev/null 2>&1 &
72 done
73
74 #Funcion de stop until proceso finalice del todo
75 for c in ${container[@]}; do
76     function_Endding_Procces ${c}
77 done
78
79 echo "EM stop...."; echo ""
80 curl -s http://localhost:6000/api/stop | jq . > "${fem}"
81 docker exec scaphandre sh -c "pkill -f scaphandre"
82
83 # ALMACENAMIENTO Y PROCESAMIENTO DE DATOS
84
85 # Get data from scaphandre
86 docker cp scaphandre:/app/out.metrics "${scapfile}"

```

```
81 # Change owner of file
82 sudo chown clouduser:clouduser "${scapfile}"
83 # Stop scaphandre container
84 docker stop scaphandre
85
86
87 exit 0
```

Listing A.4: Script en bash empleado para llevar a cabo la toma de metricas de consumo energetico con la herramienta Scaphandre

Arranque y toma de metricas con Monitor

```
1 #!/bin/bash
2
3 function sanity_check() {
4     container=$1
5     id=$(docker inspect $container 2> /dev/null | jq -r '.[0].Id' )
6     [[ "$id" == "null" ]] && return 1 || return 0
7 }
8
9 function wait_to_finish(){
10     container=$1
11     app=$2
12     echo "Waiting for $app to finish in container $container"
13     while [ -n "$(docker container top $container | grep $app)" ];
14     do
15         sleep 0.5
16     done
17 }
18
19 container="ffmpeg-encoder"
20
21 if ! sanity_check $container; then
22     echo "Container $container not running"
23     echo "Stopping the script..."
24     exit 1
25 fi
26
27 repetition=$1
28
29 expName="ffmpeg-cpu"
30
31 folder="data/${expName}/rep${repetition}"
32 mkdir -p $folder
33
34 fname=$expName
35
36 # Names of the files
37 infoFile=${folder}/${fname}-info.txt
38 globalFile=${folder}/${fname}-global.json
39 csvFile=${folder}/${fname}-samples.csv
```



```

39
40 # Bit rate
41 br=6
42
43 uid=$(uuid)
44
45 payload='{ "workload_id": "'$uid'",
46           "container": "'$container'",
47           "gpu_indices": [0],
48           "persist_data": 0}'
49
50
51 # Global energy capture
52 curl -s http://localhost:6000/api/start
53
54 # Petición de monitorización
55
56 response=$(curl -s -H "Content-Type: application/json" -X POST -d "
57   $payload" http://localhost:5000/api/monitor)
58 echo "Response -- ${response}"
59
60 id=$(echo "$response" | jq -r '.id')
61
62 echo "Monitoring container $id"
63
64 # Ejecución de tarea de codificación de vídeo
65 #libx264 hvenc_264
66 workload_command="sleep 1.5; ffmpeg -y -i /config/
67   SkLab_3840x2160_crf0.mp4 -c:v libx264 -b:v 4M -vf scale=1280:720
68   -c:a copy /config/output.mp4 ; sleep 1.5"
69
70 docker exec -d $container sh -c "$workload_command"
71 sleep 5
72
73 # Espera activa
74 wait_to_finish $container ffmpeg
75
76 output="$(curl -s -X DELETE http://localhost:5000/api/monitor?id=
77   $id)"
78
79 # Global energy capture stop
80 curl -s http://localhost:6000/api/stop > ${globalFile}
81
82 echo "Command: $workload_command" > ${infoFile}
83 docker exec $container sh -c "ls -al /config/output.mp4" >> ${
84   infoFile}
85 echo "Output: $output" >> ${infoFile}
86 docker exec $container sh -c "ffprobe -v error -select_streams v -
87   of default=noprint_wrappers=1:nokey=1 -show_entries stream=
88   r_frame_rate,width,height,duration,bit_rate -of default=

```

```

84     noprint_wrappers=1 /config/output.mp4" >> ${infoFile}
85 curl -s http://localhost:5000/api/monitor/workload/$uid/samples > ${csvFile}

```

Listing A.5: Script en bash empleado para llevar

```

1  #!/bin/bash
2
3  sanity_check() {
4      local container="$1"
5
6      id=$(docker inspect "$container" 2>/dev/null | jq -r '.[0].Id')
7
8      [[ -n "$id" && "$id" != "null" ]]
9  }
10
11 wait_to_finish() {
12     local container="$1"
13     local app="$2"
14
15     echo "Waiting for $app to finish in container $container"
16
17     while docker container top "$container" 2>/dev/null | grep -q "$app"; do
18         sleep 0.5
19     done
20 }
21
22 container1="ffmpeg-encoder-limited-1"
23 container2="ffmpeg-encoder-limited-2"
24
25 if ! sanity_check "$container1" || ! sanity_check "$container2";
26 then
27     echo "Some container isn't running"
28     echo "Stopping the script..."
29     exit 1
30 fi
31
32 repetition="$1"
33
34 expName="ffmpeg-2_containers-limited-3_cores"
35
36 folder="data/${expName}/rep${repetition}"
37 mkdir -p "$folder"
38
39 fname="${expName}"
40
41 infoFile="${folder}/${fname}-info.txt"
42 globalFile="${folder}/${fname}-global.json"
43
44 csvFile1="${folder}/${fname}-${container1}-samples.csv"
45 csvFile2="${folder}/${fname}-${container2}-samples.csv"

```

```

45
46 # Bit rate
47 br=6
48
49 uid1=$(uuid)
50 uid2=$(uuid)
51 echo "UUID 1 --> ${uid1} || UUID2 --> ${uid2}"
52
53 payload1=$(cat <<EOF
54 {
55     "workload_id":"$uid1",
56     "container":"$container1",
57     "gpu_indices":[0],
58     "persist_data":0
59 }
60 EOF
61 )
62
63 payload2=$(cat <<EOF
64 {
65     "workload_id":"$uid2",
66     "container":"$container2",
67     "gpu_indices":[0],
68     "persist_data":0
69 }
70 EOF
71 )
72
73 # Global energy capture
74 curl -s http://localhost:6000/api/start
75
76 # Monitorización contenedor 1
77 response1=$(curl -s \
78     -H "Content-Type: application/json" \
79     -X POST \
80     -d "$payload1" \
81     http://localhost:5000/api/monitor)
82
83 id1=$(echo "$response1" | jq -r '.id')
84
85 echo "Response -- $response1"
86 echo "Monitoring container $container1 (id=$id1)"
87
88 # Monitorización contenedor 2
89 response2=$(curl -s \
90     -H "Content-Type: application/json" \
91     -X POST \
92     -d "$payload2" \
93     http://localhost:5000/api/monitor)
94
95 id2=$(echo "$response2" | jq -r '.id')
96

```

```

97 echo "Response -- $response2"
98 echo "Monitoring container $container2 (id=$id2)"
99
100 # Ejecución de tarea
101 workload_command="sleep 1.5; ffmpeg -y -i /config/
    SkLab_3840x2160_crf0.mp4 -c:v libx264 -b:v 4M -vf scale=1280:720
    -c:a copy /config/output.mp4 ; sleep 1.5"
102
103 docker exec -d "$container1" sh -c "$workload_command"
104 docker exec -d "$container2" sh -c "$workload_command"
105
106 sleep 2
107
108 # Espera activa
109 wait_to_finish "$container1" ffmpeg
110 output1=$(curl -s -X DELETE "http://localhost:5000/api/monitor?id=$
    {id1}")
111
112 wait_to_finish "$container2" ffmpeg
113 output2=$(curl -s -X DELETE "http://localhost:5000/api/monitor?id=$
    {id2}")
114
115 # Global energy capture stop
116 curl -s http://localhost:6000/api/stop > "$globalFile"
117
118 # Guardar información
119 {
120     echo "Command: $workload_command"
121     echo
122
123     echo "Container: $container1"
124     docker exec "$container1" sh -c "ls -al /config/output.mp4"
125     echo "Output: $output1"
126     docker exec "$container1" sh -c \
127         "ffprobe -v error \
128         -select_streams v \
129         -show_entries stream=r_frame_rate,width,height,duration,
130         bit_rate \
131         -of default=noprint_wrappers=1 \
132         /config/output.mp4"
133
134     echo
135     echo "Container: $container2"
136     docker exec "$container2" sh -c "ls -al /config/output.mp4"
137     echo "Output: $output2"
138     docker exec "$container2" sh -c \
139         "ffprobe -v error \
140         -select_streams v \
141         -show_entries stream=r_frame_rate,width,height,duration,
142         bit_rate \
143         -of default=noprint_wrappers=1 \
144         /config/output.mp4"

```

```

143
144 } > "$infoFile"
145
146 # Descargar muestras
147 echo "---1--- $csvFile1 ----"
148 curl -s \
149     "http://localhost:5000/api/monitor/workload/${uid1}/samples" \
150     > "$csvFile1"
151
152 echo "---2--- $csvFile2 ----"
153 curl -s \
154     "http://localhost:5000/api/monitor/workload/${uid2}/samples" \
155     > "$csvFile2"

```

Listing A.6: Script en bash empleado para llevar a cabo la toma de metricas de consumo energetico sobre dos contenedor simultaneamente con la herramienta Monitor

Procesamiento de Metricas

```

1  #!/bin/bash
2
3  if [ $# -lt 1 ]; then
4      echo "Numero de argumento incorrectos :- ("
5      echo "***** Debes de indicar *****"
6      echo "**\n Numero_Iteracion      **"
7      exit 1
8  fi
9
10 # function get_name_containers(){
11 #     containers_list=$1
12 #     name_all_containers_list=$(docker ps -a --format=json | jq
13 # -r '.Names' | tr -d '/' ))
14 #     container_name_used_list=()
15 #     for cont_all in "${name_all_containers_list[@]}"; do
16 #         for cont in "${containers_list[@]}"; do
17 #             cont_all_id=$(docker inspect $i | jq -r '[][.Id]')
18 #             if [[ $cont == $cont_all_id ]]; then
19 #                 container_name_used+=($cont_all)
20 #             fi
21 #         done
22 #     done
23 #     echo "${container_name_used_list[@]}"
24 # }
25
26 iteracion=$1
27 herramienta="kepler"
28 dir_root="$HOME/Documentos/Experiments/0_test_herramientas"
29 sub_dir="${dir_root}/${herramienta}/full" # Ubicacion del ARCHIVO
30 kind_iter="full"
31

```

```

32 fem="${dir_root}/em/full/em_${herramienta}_${iteracion}.json" #
    Ubicacion archivos finales em
33 kepler_raw_results="${sub_dir}/zdirty_raw_${herramienta}_${
    kind_iter}_${iteracion}.metrics" # RAW Data
34 final_results_csv="${sub_dir}/${herramienta}_10_it_${kind_iter}.csv
    " #Nombre del arcohivo final resultante
35 #container=$(cat ${kepler_raw_results} | grep
    kepler_container_cpu_joules_total | grep -v '#' | grep 'zone="
    core"' | cut -d ',' -f1 | cut -d '=' -f2 | tr -d '"' | sort -u))
36 #container=$(get_name_containers $container))
37 container=("ffmpeg-encoder")
38
39 if [[ ! -f "${final_results_csv}" ]]; then
40     echo "Creating FINAL RESULTS FILES ${final_results_csv}...."
41     echo "IT,container,em_host,gpu_process_joules,
    core_process_joules,dram_process_joules,package_process_joules,
    total_process_joules,total_node_joules" > "${final_results_csv}"
42
43 fi
44
45 echo "Processing Data...."
46
47 for c in ${container[@]}; do
48     id=$(docker inspect ${c} | jq -r '.[0].Id')
49     # Porcentaje de Global segun EM
50     em_host=$(cat $fem | jq -r '.total')
51     exp_time=$(cat $fem | jq -r '.time')
52     # Consumo del Host
53     gpu_process="$(cat ${kepler_raw_results} | grep "
    kepler_node_gpu_watts" | grep -v "#" | cut -d ' ' -f3 | tr -d '
    ')"
54     gpu_process_joules="$(echo "${gpu_process}*${exp_time}" | bc -l
    )"
55     # Consumo CORE Process
56     core_process="$(cat ${kepler_raw_results} | grep -i $id | grep
    -v "#" | grep "kepler_container_cpu_joules_total" | grep 'zone
    ="core"' | cut -d ' ' -f2 | tr -d " ")"
57     # Consumo DRAM Process
58     dram_process="$(cat ${kepler_raw_results} | grep -i $id | grep
    -v "#" | grep "kepler_container_cpu_joules_total" | grep 'zone
    ="dram"' | cut -d ' ' -f2 | tr -d " ")"
59     # Consumo PKG Process
60     package_process="$(cat ${kepler_raw_results} | grep -i $id |
    grep -v "#" | grep "kepler_container_cpu_joules_total" | grep '
    zone="package"' | cut -d ' ' -f2 | tr -d " ")"
61     # Consumo Total del Processo
62     total_process="$(echo "${dram_process}+${package_process}" | bc
    -l)"
63     #Consumo Total del Nodo/Host
64     total_node_joules_pkg="$(cat ${kepler_raw_results} | grep -v "#"
    " | grep "kepler_node_cpu_active_joules_total" | grep 'zone="
    package"' | cut -d ' ' -f2 | st-console --sum)"

```

```

65     total_node_dram="$(cat ${kepler_raw_results} | grep -v "#" |
    grep "kepler_node_cpu_active_joules_total" | grep 'zone="dram"'
    | cut -d ' ' -f2 | st-console --sum)"
66     total_node_joules="$(echo "${total_node_joules_pkg}+${
    total_node_dram}" | bc -l)"
67
68     echo "${iteracion},${c},${em_host},${gpu_process},${
    core_process},${dram_process},${package_process},${total_process
   },${total_node_joules}" >> "${final_results_csv}"
69
70     echo "${c} -- Resultados de Iteracion ${iteracion} almacenados
    en ${final_results_csv}"
71 done

```

Listing A.7: Script en Bash que procesa los resultados de la experimentacion con la herramienta Kepler y los almacena en CSV

```

1  #!/bin/bash
2
3  if [ $# -lt 1 ]; then
4      echo "Numero de argumento incorrectos :- ("
5      echo "***** Debes de indicar *****"
6      echo "**\n Numero_Iteracion      **"
7      exit 1
8  fi
9
10 iteracion=$1
11 herramienta="scp"
12 dir_root="$HOME/Documentos/Experiments/0_test_herramientas"
13 sub_dir="${dir_root}/${herramienta}/cpu"
14 kind_iter="cpu"
15
16 scapfile="$sub_dir/zdirty_raw_${herramienta}_${kind_iter}_${
    iteracion}.metrics" # RAW Data
17 final_results_csv="$sub_dir/${herramienta}_10_it_${kind_iter}.csv"
    #Nombre del arcohivo final resultante
18 fem="${dir_root}/em/cpu/em_${herramienta}_${kind_iter}_${iteracion
    }.json"
19 codec="libx264" #libx264 h264_nvenc
20 pids_list=$(cat ${scapfile} | grep
    scaph_process_power_consumption_microwatts | grep "${codec}" |
    grep -v dockerexec | cut -d ' ' -f4 | jq -r ".pid" | sort -u)
21
22 if [[ ! -f "$final_results_csv" ]]; then
23     echo "IT,pid,em_host,total_energy_host,cpu_usage_percentage,
    total_process_joules" > "$final_results_csv"
24 fi
25
26 echo "Processing Data...."
27 #Determinar Numero de procesos lanzados
28
29 for pid in ${pids_list[@]}; do
30     # Porcentaje de Global segun EM

```

```

31   em_host=$(cat $fem | jq ".total")
32   # Porcentaje de CPU Usado en proceso
33   cpu_usage=$(cat ${scapfile} | grep
scaph_process_cpu_usage_percentage | grep "${codec}" | grep "${
pid}" | grep -v dockerexec | cut -d '{' -f1 | cut -d '=' -f2 | tr
-d ' ' | st-console --mean)
34   # Consumo del Host
35   scp_host=$(cat ${scapfile} | grep scaph_host_power_microwatts |
cut -d '{' -f1 | cut -d '=' -f2 | tr -d ' ' | xargs -i echo "
1*{}/1000000" | bc -l | st-console --sum)
36   # Consumo del Proceso FFMPEG
37   scp_process=$(cat ${scapfile} | grep
scaph_process_power_consumption_microwatts | grep "${codec}" |
grep "${pid}" | grep -v dockerexec | cut -d '{' -f1 | cut -d '='
-f2 | tr -d ' ' | xargs -i echo "1*{}/1000000" | bc -l | st-
console --sum )
38
39   echo "${iteracion},${pid},${em_host},${scp_host},${cpu_usage},${
scp_process}" >> "${final_results_csv}"
40
41   echo "Resultados de proceso ${pid} en Iteracion $iteracion
almacenados en $final_results_csv"
42 done

```

Listing A.8: Script en Bash que procesa los resultados de la experimentacion con la herramienta Scaphandre y los almacena en CSV

```

1  #!/bin/bash
2  base=$(pwd)/data
3  case="$2"
4  sub_cont="ffmpeg-$case/rep$1"
5  workdir="${base}/${sub_cont}"
6  path_file=$(ls ${workdir} | grep "csv")
7  global_path_file="${workdir}/${ls ${workdir} | grep "json")"
8  global_info=${workdir}/${ls ${workdir} | grep "txt")
9  final_workdir="ffmpeg-$case-processed.csv"
10
11
12
13  if [[ ! -f ${final_workdir} ]]; then
14      head="tool,case,em_host,gpu_process_joules,node_total_joufig:
exp1-kepler-csv-headerles"
15      for i in "${!path_file[@]}";do
16          indice=$i
17          indice=$((indice + 1))
18          head="$head,process${indice}_energy_joules"
19      done
20      echo "${head}" >> ${final_workdir}
21  fi
22
23  e_pkg=$(cat "${workdir}/${path_file[0]}" | cut -d ';' -f4 | sed
'1d' | st-console --sum)
24  e_drm=$(cat "${workdir}/${path_file[0]}" | cut -d ';' -f10 |

```



```

25 sed '1d' | st-console --sum)
    gpu_energy=$(cat ${global_info} | grep Output | sed 's/^
Output: //' | jq -r '.energy_per_gpu.[0]')
26 total_sum=$(echo "${e_pkg}+${e_drm}" | bc -l)
27 em_total_energy=$(cat "${global_path_file}" | jq -r '.total')
28 result="monitor,${case},${em_total_energy},${gpu_energy[0]},{
total_sum}"
29
30 for pf in "${path_file[@]";do
31     c_e_pkg=$(cat "${workdir}/${pf}" | cut -d ';' -f9 | sed '1d'
| st-console --sum)
32     c_e_drm=$(cat "${workdir}/${pf}" | cut -d ';' -f14 | sed '1d'
| st-console --sum)
33     total_sum_cont=$(echo "${c_e_pkg}+${c_e_drm}" | bc -l)
34     result="${result},${total_sum_cont}"
35 done
36
37 echo "${result}" >> ${final_workdir}
38 echo "Iteracion $1 procesada ==> ${result}"

```

Listing A.9: Script en Bash que procesa los resultados de la experimentacion con la herrameinta Monitor y los almacena en CSV

Obtencion de graficas Comparativas

```

1 #!/bin/bash
2
3 function sanity_check() {
4     container=$1
5     id=$(docker inspect $container 2> /dev/null | jq -r '.[0].Id' )
6     [[ "$id" == "null" ]] && return 1 || return 0
7 }
8
9 function wait_to_finish(){
10     container=$1
11     app=$2
12     echo "Waiting for $app to finish in container $container"
13     while [ -n "$(docker container top $container | grep $app)" ];
14     do
15         sleep 0.5
16     done
17 }
18
19 container="ffmpeg-encoder"
20
21 if ! sanity_check $container; then
22     echo "Container $container not running"
23     echo "Stopping the script..."
24     exit 1
25 fi
26
27 repetition=$1

```

```
27
28 expName="ffmpeg-cpu"
29
30 folder="data/${expName}/rep${repetition}"
31 mkdir -p $folder
32
33 fname=$expName
34
35 # Names of the files
36 infoFile=${folder}/${fname}-info.txt
37 globalFile=${folder}/${fname}-global.json
38 csvFile=${folder}/${fname}-samples.csv
39
40 # Bit rate
41 br=6
42
43 uid=$(uuid)
44
45 payload='{ "workload_id": "'$uid'",
46           "container": "'$container'",
47           "gpu_indices": [0],
48           "persist_data": 0}'
49
50
51 # Global energy capture
52 curl -s http://localhost:6000/api/start
53
54 # Petición de monitorización
55
56 response=$(curl -s -H "Content-Type: application/json" -X POST -d "
57     $payload" http://localhost:5000/api/monitor)
58
59
60 id=$(echo "$response" | jq -r '.id')
61
62 echo "Monitoring container $id"
63
64 # Ejecución de tarea de codificación de vídeo
65 #libx264 hvenc_264
66 workload_command="sleep 1.5; ffmpeg -y -i /config/
67     SkLab_3840x2160_crf0.mp4 -c:v libx264 -b:v 4M -vf scale=1280:720
68     -c:a copy /config/output.mp4 ; sleep 1.5"
69
70
71
72 docker exec -d $container sh -c "$workload_command"
73 sleep 5
74
75 # Espera activa
76 wait_to_finish $container ffmpeg
```

```

75 output="$(curl -s -X DELETE http://localhost:5000/api/monitor/?id=
    $id)"
76
77 # Global energy capture stop
78 curl -s http://localhost:6000/api/stop > ${globalFile}
79
80 echo "Command: $workload_command" > ${infoFile}
81 docker exec $container sh -c "ls -al /config/output.mp4" >> ${
    infoFile}
82 echo "Output: $output" >> ${infoFile}
83 docker exec $container sh -c "ffprobe -v error -select_streams v -
    of default=noprint_wrappers=1:nokey=1 -show_entries stream=
    r_frame_rate,width,height,duration,bit_rate -of default=
    noprint_wrappers=1 /config/output.mp4" >> ${infoFile}
84
85 curl -s http://localhost:5000/api/monitor/workload/$uid/samples > $
    {csvFile}

```

Listing A.10: Script en Python que permite obtener graficas comparativas de consumo energetico entre las herrameintas Monitor, Scaphandre y Kepler

A.1.2. Experimentacion Fase 2

Obtencion de datos con FFPROBE

```

1 ffprobe -v error -select_streams v:0 -count_frames -
  show_entries stream=nb_read_frames -of csv=p=0 VIDEO.mp4

1 ffprobe -v error -select_streams v:0 -show_entries stream=
  bits_per_raw_sample,pix_fmt -of default=noprint_wrappers=1:nokey
  =1 VIDEO.mp4

```

Listing A.11: Proundidad de color con ffprobe

Codigo Python para Calculo y Visualizacio de metricas SITI

CODIGO PYTHON SITI

CODIGO PYTHON PLOTTER

Bibliografía

- [1] Rupinder Kaur, Gurjinder Kaur, and Major Singh Goraya. A workflow scheduling algorithm with task classification for reducing energy consumption and makespan in cloud. *Sustainable Computing: Informatics and Systems*, 50:101323, June 2026.
- [2] Samira Afzal, Narges Mehran, Zoha Azimi Ourimi, Farzad Tashtarian, Hadi Amirpour, Radu Prodan, and Christian Timmerer. A Survey on Energy Consumption and Environmental Impact of Video Streaming, January 2024. arXiv:2401.09854 [cs.MM].
- [3] Kashif Nizam Khan, Mikael Hirki, Tapio Niemi, Jukka K. Nurminen, and Zhonghong Ou. RAPL in Action: Experiences in Using RAPL for Power Measurements. *ACM Trans. Model. Perform. Eval. Comput. Syst.*, 3(2):9:1–9:26, March 2018.
- [4] Michael Armbrust, Armando Fox, Rean Griffith, Anthony D. Joseph, Randy Katz, Andy Konwinski, Gunho Lee, David Patterson, Ariel Rabkin, Ion Stoica, and Matei Zaharia. A view of cloud computing. *Commun. ACM*, 53(4):50–58, April 2010.
- [5] Ashkan Safari, Hoda Sorouri, Afshin Rahimi, and Arman Oshnoei. A systematic review of energy efficiency metrics for optimizing cloud data center operations and management. *Electronics*, 14(11):2214, May 2025.
- [6] Electricity 2026 – analysis.
- [7] Global energy review 2026 – analysis.
- [8] Share:. Cop2030 presidency releases executive report and outlines next steps to accelerate global climate implementation.
- [9] Intel core i7-4790k processor (8m cache, up to 4.40 ghz) - product specifications.
- [10] nvidia.com. https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/documents/75509_DS_NV_Quadro_M4000_US_NV_HR.pdf. [Accessed 05-07-2026].
- [11] nvidia-smi-documentation.
- [12] Installing the nvidia container toolkit — nvidia container toolkit.
- [13] Kepler.
- [14] Introduction - scaphandre documentation.
- [15] Xubuntu – Releases.
- [16] Kernel modules — nvidia nriver installation guide.
- [17] Debian – versiones de debian.

-
- [18] ffmpeg - linuxserver.io.
 - [19] Home - linuxserver.io.
 - [20] Choose live encoder settings, bitrates, and resolutions.
 - [21] 4k Video.
 - [22] P.910:subjective video quality assessment methods for multimedia applications.
 - [23] siti-tools: Functions to calculate spatial information / temporal information according to itu-t.910.